

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITY OF MODENA AND REGGIO EMILIA

"Enzo Ferrari" Department of Engineering

Master's Degree in Artificial Intelligence Engineering (LM-32)

Continual Learning Framework for Opponent Detection in Autonomous Car Races

Supervisor:
Paolo Burgio
Co-supervisors:
Alessandro Capotondi
Francesco Moretti

Candidate:
Tommaso Mortara
Student ID 206388

ACADEMIC YEAR 2025/2026

"Vietato calpestare i sogni"

– Nonna Anna Rosa

Contents

1 Abstract **1**

2 Introduction **3**

2.1 Contributions 4

2.2 Outline 4

3 Background **6**

3.1 Autonomous Driving Systems 6

3.2 RoboRacer (F1TENTH) Platform 8

3.3 Deep Learning and Computer Vision 8

3.4 Object Detection 10

3.4.1 Introduction 10

3.4.2 Object Detection Architectures 11

3.5 Dataset 11

3.6 Continual Learning 12

3.6.1 The Concept 12

3.6.2 The Problem 13

3.7 Evaluation Metrics 14

3.7.1 Object Detection Metrics 14

3.7.2 Continual Learning Metrics 16

4 State of the Art **18**

4.1 Object Detection Architectures Evolution 18

4.1.1 Two-stage Detectors 18

4.1.2 Single-stage Detectors 19

4.1.3 Comparison 19

4.2 The YOLO Family 20

4.2.1 Evolution of the Architecture 20

4.2.2 The Chosen Model: YOLO26 . . . 21
4.3 Continual Learning Strategies: A Comparative Overview . . . 22
4.3.1 Regularization-based Methods . . . 22
4.3.2 Replay-based Methods . . . 23
4.3.3 Parameter Isolation Methods . . . 23
4.4 Continual Learning Object Detection . . . 24
4.5 Existing CL Frameworks . . . 26

5 Contributions **27**
5.1 Overview . . . 27
5.2 Custom Dataset . . . 27
5.2.1 Methodology . . . 27
5.2.2 Implementation . . . 28
5.2.3 Dataset Composition . . . 30
5.3 Continual Learning Framework . . . 31
5.3.1 Overview . . . 31
5.3.2 Preprocessing . . . 32
5.3.3 Benchmarking . . . 38

6 Results **43**
6.1 Research Questions . . . 43
6.2 Benchmarks and Experiments Set-Up . . . 44
6.3 Results . . . 46
6.4 Discussion . . . 51
6.4.1 Forgetting . . . 53
6.4.2 Generalization . . . 53
6.4.3 Specialization . . . 54
6.4.4 Conclusions . . . 54
6.5 Sequential Training . . . 56

7 Future Works **59**

7.1 Labelling 59

7.2 On-edge Deployment 60

8 Conclusions **61**

References **63**

List of Figures

3.1 *Three-layer paradigm of automation* 7
3.2 *Example of a LiDAR point cloud.* 7
3.3 *Example of a RoboRacer (F1TENTH) vehicle.* 9
3.4 *Object detection as union of classification and localization* 11
3.5 *Different types of incremental learning* 13
3.6 *Example of forgetting in Continual Learning* 14
3.7 *Intersection over Union (IoU) definition* 15

5.1 *Example of a RealSense Depth Camera D435i.* 28
5.2 *Logical flow of the dataset loading process.* 37
5.3 *The benchmarking pipeline.* 42

6.1 *Training curves (mAP50-95 and FAR vs epochs) for the base model under different data-split configurations.* 47
6.2 *Training curves (mAP50-95 and FAR vs epochs) for the three adaptation strategies.* 49
6.3 *Comparison of the three methods across the three benchmarks: mAP50-95 (left column) and FAR (right column), for* `Forgetting`, `Generalization` *and* `Specialization` *(rows).* 50
6.4 *Graphical demonstration of generalization and specialization with respect to the required time for training.* 55
6.5 *Graphical explanation of the sequential training mechanism.* 56
6.6 *Graphical explanation of the sequential results.* 57

List of Code

5.1 *ImageSample dataclass definition* 32
5.2 *Dataset dataclass definition* 32
5.3 *Basic Loader Interface* 33
5.4 *Image loading logic* 33
5.5 *datasets.yaml generic entries* 34
5.6 *Centralized addition of datasets* 34
5.7 *experiments.yaml generic entries* 35
5.8 *Default hyperparameters for training* 36
5.9 *benchmarks.yaml generic entry* 39
5.10 *example of dumps of predicted detections* 39
5.11 *example of dump's metadata* 39

1. Abstract

Autonomous Driving Systems stand on their ability to understand the world around them, through Perception algorithms: in this work, it has been investigated how Object Detection can impact said systems, leveraging images captured through the on-board camera of the RoboRacer platform.

The realization of this system has been slowed down by two main problems: first, the lack of publicly available datasets to let the model train and learn on, and second, the way in which this data can be acquired. This last phenomenon is problematic, because it goes against one of the main requirements for data that has to be used in Convolutional Neural Networks: the i.i.d. (Independent and Identically Distributed) assumption. New data is usually available in sessions and streams of images, all linked by the environment it is recorded in. All these requirements move toward an approach that differs from the classical learning, called Incremental or Continual Learning.

The first part of the work is represented by the creation of a novel and customizable dataset, well suited for welcoming new sessions and streams of data even after it has already been trained. This is not however all this thesis contributes to: a complete framework has been built that allows the actual addition of new data and then handles it in such a way to let any user of the project contribute and also design new experiments to evaluate the behavior of a network when trained on a different composition of available data.

This last situation is the most similar to the Continual Learning one: this type of Learning's main problem is Catastrophic Forgetting, that is the tendency of a model to forget old tasks and environments when tuned on new ones.

Based on this, the work includes a section where the framework is employed to evaluate the performance of models when they have to handle new data. Three main situations have been investigated, like a full retraining, starting from a YOLO26n pre-trained model, a naive fine tuning and one of the main approaches to limit said forgetting: the experience replay. It aims to add part of the old data to the new training set, in order not to let the resulting model forget what it has already previously learned. All these experiments have been conducted with different amounts of data, with the purpose of understanding the impact of data availability on model performance, and also each run has been repeated multiple times to avoid the risk of depending on luck.

The results show that there is not a go-to approach that is the best in every situation, but the choice depends on the requirements of the task. If the end goal is to have the model that best retains old information, while being able to address the new domain, the application of Experience Replay in training is the correct approach, but if it is important to have the best possible performance, the complete Joint Retrain has the best absolute values. When considering the task of specializing on the new domain, the results are very close on almost every configuration, so the choice is almost irrelevant. Adding training times to the considered metrics, the length of it for every model can be taken into consideration, and for different applications, a trade-off between mAP50-95, FAR and efficiency gives different results. For the last research question in particular, Joint Retrain employs 27 times the time of Fine Tuning, while giving almost the same mAP50-95.

2. Introduction

The work that is about to be presented finds its roots in the basic necessity to create a perception system able to understand the presence of opponents' racecars, or a lack of them on the RoboRacer platform. This was required as the development of an autonomous racing software stack was being created and refined in order to attend an actual competition. In order to complete this task, the required system was one that leverages object detection possibilities not as an end in themselves, but by considering and paying close attention to the specific use case's requirements and limitations.

One of these was encountered as a first step in the approach to the project: the lack of publicly available data. By scouting the internet it became clear that there were no datasets able to fulfill the project requirements to become standard training ground and evaluation benchmark to create and possibly compare the developed solution with others in literature: not even a large group of images, maybe still to label, was found. All that was available was a small group of labeled images, already augmented and not very suitable for the creation from an empty base of a viable model. The only approach was to develop a custom dataset by organizing real data gathering sessions, together and with the help of the HiPeRT team, from the running racecars, by employing the RealSense Camera sensor mounted on top of the car. These images needed to be labeled and fed through the selected network, a YOLO26n model.

During these data sampling sessions, it was considered that a lot of real-world applications present similar limitations and requirements, especially the lack of a fixed dataset and the availability of data mainly in streams with close semantic relationships, all coming in subsequent iterations. In these scenarios there still is a need of trying to include new data to continuously improve performance: this general problem takes the name of Continual Learning.

It was decided to tackle this landscape with an effort in trying to build a framework that can help to perform actual training and develop research and analysis on real data and results. A framework has been developed with multiple purposes: new data inclusion in the existing dataset; common preprocessing, loading and virtualization on the actual training pipeline; experiments and hyperparameters tuning. All these steps are needed to perform the computation of weights to have some models to perform analysis of results on, and the proposed solution aims to abstract and make

these technical steps transparent to any successive user of the framework.

Finally, it has been decided to test the abilities of said system with the creation of multiple research questions that we tried to give an answer to in the precise context that has been just discussed. The general doubt was about what could be the best approach in the introduction of a new subset of data into the existing batch, and the considered solutions were: Joint Retrain, that is the training starting from a YOLO26n model of the entire dataset in which the new session is included; Naive Fine Tuning, that executes a lighter training of the last computed model on just the new set of data; finally Fine Tuning with Experience Replay, that is the same as the last, with the difference that all new data is fed through with the addition of a percentage of the base batch samples.

2.1 Contributions

This thesis' contributions can be summarized in the following list:

- **Custom Dataset Creation**: the development of an easy-to-improve dataset to employ when developing an object detection system in the context of the RoboRacer platform.
- **Continual Learning Framework**: the creation of an easy-to-use tool that can be reshaped and reused towards various applications of Computer Vision.
- **Evaluation on Three Research Questions**: the employment of said framework to give an answer to useful questions in the broader field of Continual Learning for Object Detection.
- **Sequential Training Research**: the investigation of Continual Learning methods' behavior in situations that our previous application was not able to tackle.

2.2 Outline

The work has been organized in this way:

- **Background**: a first approach to the core concepts of the thesis, with definitions and explanations useful to understand more deeply the rest of the work
- **State of the Art**: this is the review of the current state of research on the core fundamental concepts for the thesis and of similar project, explaining the gap that this work aims to fill.

- **Contributions**: a methodological and implementational explanation of the approach toward the development of the Custom Dataset and the entire framework.
- **Results**: an in-depth analysis of the obtained experimental results.
- **Future Works**: an overview of possible work that still has to be done in order to deepen some of the concepts of the thesis or to refine the end product.
- **Conclusions**: a final chapter in which to summarize the work that has been done and its significance.

3. Background

Before diving deeper into the core of this thesis, it will be helpful to introduce some of the core concepts and technologies on which this work is built. The chapter opens with the Autonomous Driving Systems' three-layer paradigm, that has been the main foundation of automation and robotics for quite some time; it then covers an introduction to the concepts of Deep Learning and Computer Vision, leading to an essential overview of Object Detection, the main building block to the project that this report describes. Finally, it defines the notions of dataset, and Continual Learning, both necessary for a deep understanding of the following work.

3.1 Autonomous Driving Systems

The creation and development of Autonomous Driving Systems is a complex task which involves the integration and collaboration of many different technologies and areas of expertise. Modern autonomous systems are commonly designed to follow a three-layer architecture [29]:

- **Perception**: This layer aims to retrieve the raw sensor data in order to comprehend the vehicle's surrounding environment and how it is interacting with it. This layer executes mainly motion estimation, environmental segmentation, and detection of obstacles such as other vehicles, people or generic objects. Since it involves processing different sensors' data, this is usually one of the most computationally heavy steps of the system, but the high-level data inferred during this step is usually a fundamental part of all the subsequent steps in the computation, by effectively representing the system's sensorial receptors.
- **Decision/Planning**: This layer can be interpreted as the autonomous system's brain, because it is tasked to work on the perception data to generate high-level objectives. This step aims to accomplish a pre-determined task. This could involve state machines to describe the system's behaviour and also complex motion planning techniques. In an autonomous racing scenario, the planning layer could be tasked, by the collection of an opponent's detection and localization, with the generation of a path to overtake said race car, without it intersecting any obstacle that may be in the way.

- **Control**: The final control layer is tasked to manipulate the system's actuators (engines, steering, etc.) in order to accomplish the goal of the planning layer. This involves both the low-level control of the actuators but also a control loop on higher-level information from the planning layer. This layer behaves just like the muscular system in the human body.

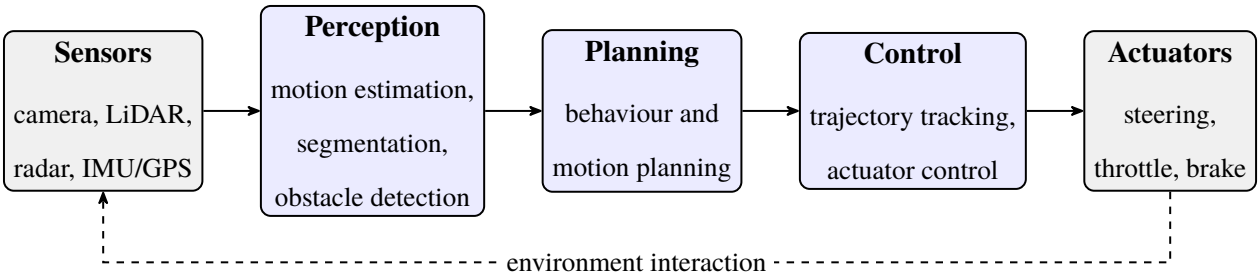


Figure 3.1: *Three-layer paradigm of automation*

Modern autonomous driving solutions require a robust and accurate perception of the surrounding environment as a fundamental building block to all subsequent decision and control algorithms. This perception can be carried out leveraging a wide variety of different sensors like camera, LiDAR, radar or ultrasonic sensors. A LiDAR (Light Detection and Ranging) estimates the distance of the surrounding surfaces by emitting laser pulses and measuring their time of flight.

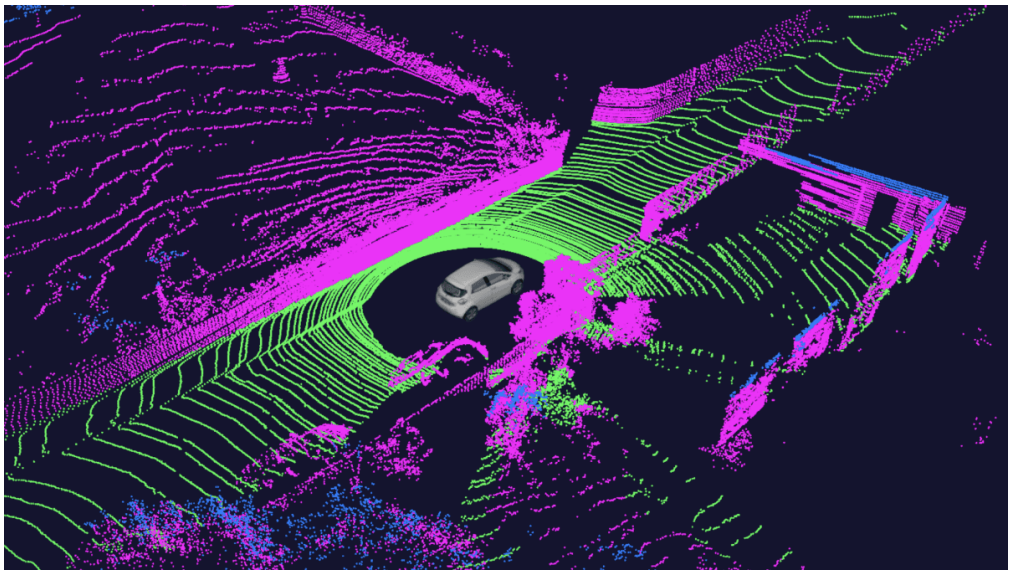


Figure 3.2: *Example of a LiDAR point cloud.*

While such a geometric reconstruction is extremely precise, it is inherently sparse and devoid

of semantic content: a point cloud describes where surfaces are, but not what they represent, and it cannot capture appearance cues such as colours, textures, lane markings or traffic signs. The camera fills exactly this gap, being the only sensor able to provide the rich semantic and appearance information on which many higher-level driving tasks rely [9]. Since the interpretation of raw images is a challenging and error-prone process for modern safety-relevant systems, it is generally entrusted to dedicated deep learning models. Camera-based perception, which is the main case study of the thesis, is the task of extracting from images a structured understanding of the scene, such as the detection and localisation of the surrounding agents and obstacles.

3.2 RoboRacer (F1TENTH) Platform

As Charles et al. [3] state, Autonomous driving technology has advanced remarkably in recent years, driving progress in artificial intelligence, robotics, and control systems. A key factor in this advancement is the ability to rigorously test and evaluate control algorithms, sensing technologies, and real-time decision-making in dynamic yet controlled environments. Platforms designed for this purpose play a vital role in bridging the gap between theoretical development and practical deployment. Among these, the RoboRacer, or F1TENTH, platform has emerged as a leading tool for advancing autonomous racing research. By providing a realistic, scalable, and cost-effective testbed, it facilitates cutting-edge research, education, and innovation in this rapidly evolving field. This platform, typically 1/10thscale autonomous vehicle system, replicates real-world challenges such as high-speed driving, trajectory planning, and real-time control. The RoboRacer platform is designed to advance research and education in autonomous driving and robotics. Guided by principles of scalability, modularity, and self-sufficiency, it offers a realistic and adaptable environment for testing, development, and innovation.

3.3 Deep Learning and Computer Vision

As said by Diwan et al. [7], Deep Learning (DL) was introduced in the early 2000s after Support vector machines (SVM), Multilayer perceptron (MLP), Artificial Neural Networks (ANN), and other neural networks became popular. It represents a subset of Machine learning (ML) which is

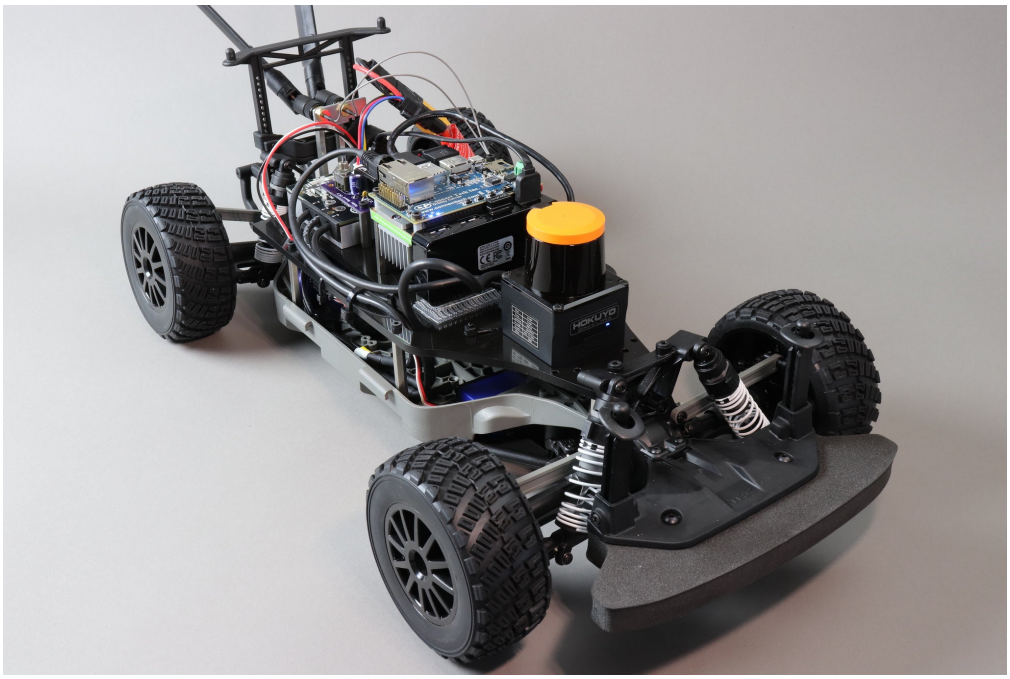


Figure 3.3: *Example of a RoboRacer (F1TENTH) vehicle.*

itself considered a subset of Artificial Intelligence (AI). During its initial conception, deep learning didn't draw much attention due to scalability and several other influential factors such as the need for huge computing power. After 2006, everything changed and it became popular as compared to its contemporary ML algorithms because of two main reasons:

- **Availability of large datasets**: the construction of large datasets such as ImageNet, COCO, and Open Images has enabled deep learning models to train on vast amounts of data, leading to improved performance and generalization.
- **Advancements in hardware**: the development of powerful GPUs and specialized hardware accelerators has significantly reduced the training time for deep learning models, making it feasible to train complex architectures on large datasets.

Computer vision is a predominant and versatile field in the current landscape of deep learning and lots of research is being carried out by various researchers and companies alike. Computer vision instructs machines to analyze and understand high-level information in visual scenarios. Its subfields include scene or object recognition, object detection, video tracking, object segmentation, pose and motion estimation, scene modeling, and image restoration.

Among these tasks, most modern computer vision systems rely on Convolutional Neural Networks (CNNs), a class of deep architectures specifically designed to process images. Through learnable filters that slide across the input, CNNs exploit the spatial structure of the data to extract a hierarchy of features, from low-level cues such as edges and textures in the early layers to increasingly abstract concepts in the deeper ones. This paradigm proved its effectiveness with the introduction of AlexNet [18], which established CNNs as the foundation of modern computer vision and of the object detectors on which this thesis builds.

3.4 Object Detection

3.4.1 Introduction

In the field of Computer Vision, Object Detection represents one of the most researched upon and important branches of study. Following the description by Aziz et al. [2], it consists of the combination of two fundamental tasks:

- **Object Classification**: Object classification is the task of assigning a single category label to an image, identifying what object it contains among a predefined set of classes, without providing any information about the object's position within the image.
- **Object Localization**: Object localization is the task of determining where an object is located within an image, typically by predicting a bounding box that tightly encloses it. This answers the spatial question, but not the semantic one.

It is a fundamental computer vision process that provides detailed semantic information of image and video. It has many applications in various fields of life, such as human behavior analysis, face recognition, image classification, medical diagnosis, and autonomous driving. Object detection has proven itself to be a quite complex task, as it requires, other than a deep research about the Deep Learning architecture used as base to build upon a working model, also a large amount of data to train the model on.

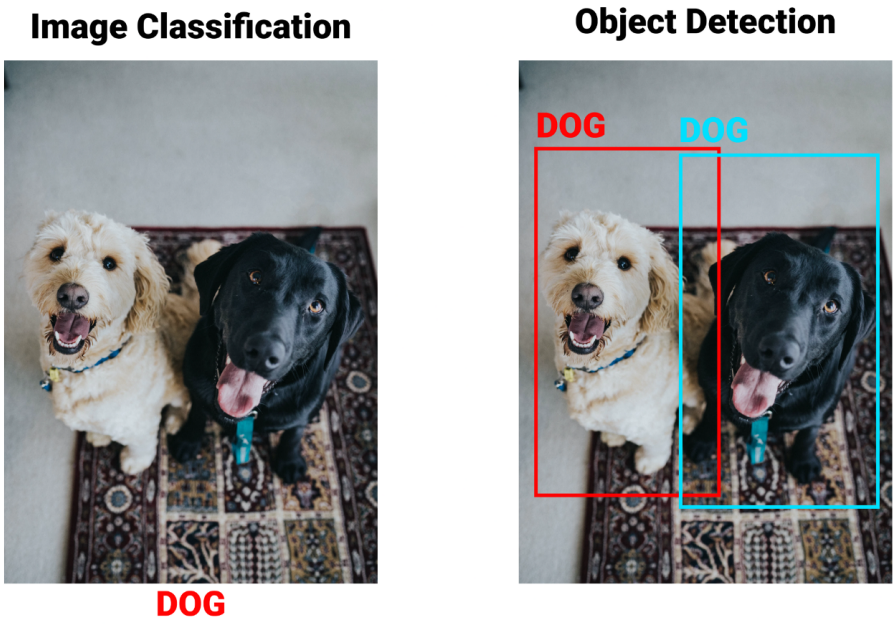


Figure 3.4: *Object detection as union of classification and localization*

3.4.2 Object Detection Architectures

Object detectors that leverage deep learning concepts, are usually grouped in two families, that differ in the way turn the input image into a set of detections [7]:

- **Two-stage detectors**: these systems, in the first step generates a set of class-agnostic region proposals, that represent areas in the image that are most likely to contain a generic object; in the second stage, classification and regression are applied to the bounding boxes through regression.
- **Single-stage detectors**: with this approach, the distinction between region proposal step and feature extraction step drops. The detection problem is resolved in a single and direct forward pass of the network, by considering the entire problem a single regression task.

3.5 Dataset

As it has already been mentioned, in order to execute correctly the object detection model training, a correctly configured dataset of images is required. With dataset we refer to a collection of images

– in the case of vision applications – that should help the base architecture to understand what and where something is in the proposed scenario. This is possible by allowing the model to learn combining multiple examples of the selected objects and the so-called ground truth. This last concept refers to the hard-coded and mainly human-annotated information of what is present in the image and where it can be found. It is possible to give these information by using bounding boxes, that highlight specific areas of the image and associate them with a specific class label.

In the contemporary landscape of computer vision, most of the projects that aim to develop an end-to-end system, start their development by leveraging existing datasets, usually publicly available and constructed thanks to the combined effort of many people and the investments of many companies. In future sections of the thesis, it will be discussed the need and the methods for a custom dataset creation, executed in order to provide a basis for the development of the entire project.

3.6 Continual Learning

3.6.1 The Concept

As it has already been mentioned, deep learning neural networks are a powerful solution to learn representations from raw data, by employing a structure of carefully placed layers, trying to mimic how the human brain is able to handle new information. They are, however, able to execute this complex task if they are presented with data that is strategically shuffled and balanced. This is not the way in which machines are able to acquire knowledge from the environment they are immersed in. In real-world applications, data usually comes in large streams and may collide with some of the main assumption about data in deep learning [13]: this is especially true in the case of autonomous driving, and specifically when developing camera-based perception. The idea of developing a perception system able to refine its knowledge and comprehension of the surrounding environment, by learning from new and specialized data, is a new and appealing concept, and steady researches have been started on novel paradigms of learning.

This section focuses on Continual Learning, also called "incremental learning" or "lifelong learning". This, as described by Menezes et al. [26], is the ability of a deep learning model to learn

consecutive tasks without forgetting the ones it has already learned. Literature [41] has, however, adopted a three task scenario convention to better explain the main branches of continual learning:

- **Task-incremental learning**: assumes the model has information about the task ID during training and testing. The situation allows for task-specific solutions.
- **Domain-incremental learning**: assumes the task ID is not given during test time, but the structure of the task is maintained. Class labels are usually kept, but the data distribution might change.
- **Class-incremental learning**: assumes the task ID is not given during test time, and model needs to infer it. In this way, the model needs to expand its range of predictions and incrementally add new classes.

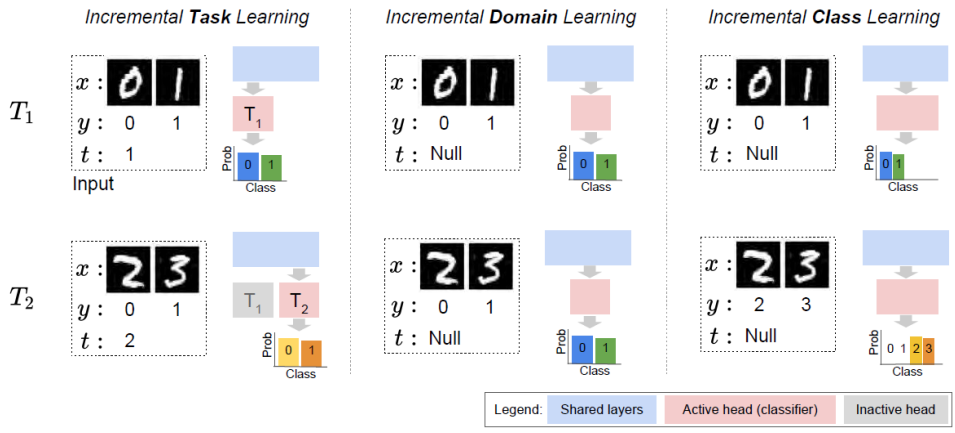


Figure 3.5: *Different types of incremental learning*

3.6.2 The Problem

The primary problem to the Continual Learning paradigm is catastrophic forgetting. This phenomenon takes place when a neural network is trained sequentially on a new task, and we can observe a fall in accuracy on previously learned tasks [11]. This is an actual expression of the trade-off between stability and plasticity of a model. Plasticity expresses the ability of the system to adapt and include new information, while a model can be called stable when it is stable enough to be able to recall what it has already learned [17]. So, the main challenge is to create a model that is able to learn new tasks without unlearning old ones.

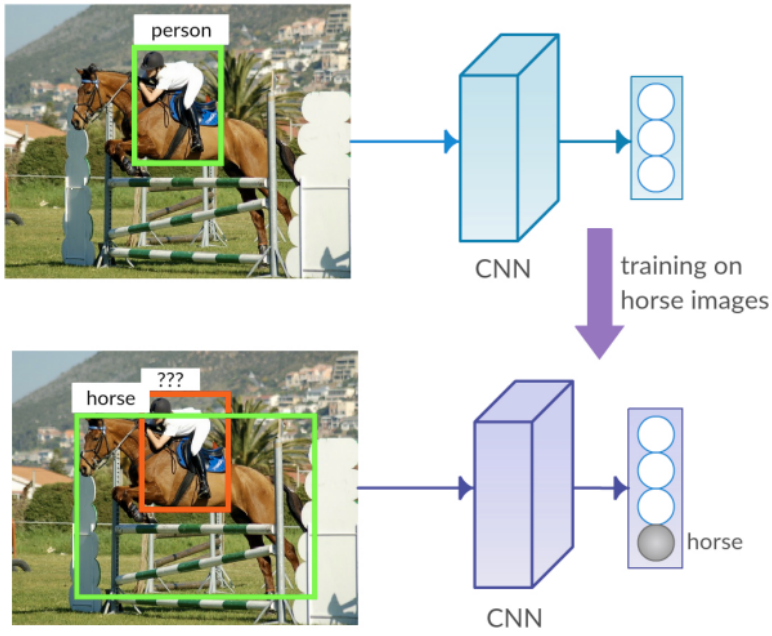


Figure 3.6: *Example of forgetting in Continual Learning*

3.7 Evaluation Metrics

In the process of evaluating the performances of a model that performs both Object Detection and Continual Learning, we need two separate groups of metrics: the first aims to measure the quality of the detection, while the second wants to comprehend the learning or forgetting or information across training iterations. This paragraph has the task to give a small introduction to the metrics that will be later used to demonstrate differences across multiple models that has been trained.

3.7.1 Object Detection Metrics

As per Padilla et al. [28], in the context of Object Detection there are some vital definition to start with:

- True positive (TP): A correct detection of a ground-truth bounding box
- False positive (FP): An incorrect detection of a nonexistent object or a misplaced detection of an existing object
- False negative (FN): An undetected ground-truth bounding box

It's important to highlight that the concept that naturally follows the aformentioned ones, True Negative (TN), exists, but it's not applicable to object detection, as we can find infinite bounding boxes that should not be detected within any given image.

There is one main approach to determine the concept of correct or incorrect detection: Intersection over Union (IoU), that measures the overlapping area between the predicted bounding box and the ground-truth bounding box, divided by the area of union between them.

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}} =$$

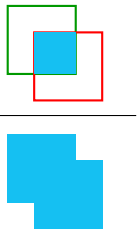


Figure 3.7: *Intersection over Union (IoU) definition*

By computing the IoU and choosing a threshold, it is possible to decide if a prediction of bounding box is correct or incorrect.

From the previous definitions stem the introduction of:

$$\text{Precision (P)} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{TP}}{\text{all detections}}$$

$$\text{Recall (R)} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{\text{all ground truths}}$$

Precision is the ability of a model to identify only relevant objects. It is the percentage of correct positive predictions. Recall is the ability of a model to find all relevant cases (all ground-truth bounding boxes). It is the percentage of correct positive predictions among all given ground truths. Both precision and recall are computed given a fixed threshold, along with their harmonic mean, that can be referred to as F1-score. By varying the threshold itself, it's possible to obtain a precision-recall curve and, computing the Area Under the Curve (AUC), to evaluate the Average Precision (AP).

AP is a valid metric to understand the accuracy of a model with respect to a single class: the mean Average Precision (mAP) is the same metric, but averaged across all predictable classes. With only one class, they represent the same value, but the one employed in this work has been this last one, just to be more consisten with the results in literature. To be more precise, it has been decided

to use the mAP version called mAP50-95, that averages the various mAP computed using different IoU thresholds, spanning from 0.50 to 0.95.

Because the base platform for the project of the thesis is an autonomous driving race car, it requires some safety-relevant specific measures, like False Alarm Rate (FAR). This metric, here defined at image level, is the fraction of negative images, that are images without any ground-truth bounding boxes, that are detected anyway as positive, over the confidence threshold:

$$\text{FAR} = \frac{\text{number of negative images with at least one detection}}{\text{total number of negative images}}$$

This is important because it represents how often the model hallucinates objects over empty images and, in the selected context, can lead to sudden maneuvers and crashes: for this reason, FAR has been considered as a fundamental metric in the evaluation of the final results.

3.7.2 Continual Learning Metrics

While measuring the performance of a model trained following the Continual Learning paradigm, it's required to consider not just a single moment in time, but multiple ones during the iterations of training phases. To compute objective reports about the performance of a model in a Continual Learning setting, the Accuracy Matrix [24] is employed. This is a comparative tool that allows to understand the evolution on the tracked metric of the model at every stage of training i, over different domains j, by filling cells as $R_{i,j}$. The rows will then represent the successive states of the model, while the columns stand for the different domains on which the model is evaluated.

As said before in subsection 3.6.2, the main objective to improve and make as good as it can be a Continual Learning model, is to optimize the trade-off between stability and plasticity. Before working in this way, it's important to understand when said trade-off gives good results, and this is possible thanks to a set of measures, each one defined as a comparison of cells in the accuracy matrix:

- **Forgetting**: as per Chaudhry et al. [4], this is defined as the performance drop on the old data caused by adapting to more recent one.
- **Backward Transfer (BWT)**: this measure is the same as Forgetting, only with the opposite sign [24]. This metric exists as, in some applications, it is more informative than the previous one.

- **Intransigence**: this represents the inability of a model to update its knowledge [4]. It is measurable as the gap between a reference model trained on all cumulate data and the continual learning one, on the same holdout set.
- **Forward Transfer (FWT)**: it is a metrics that evaluates how much the old domain already helps the new one before any adaptation [24].
- **Average performance (ACC)**: it is the mean over the domains after adaptation.

4. State of the Art

In this chapter, it is possible to find a review of the current work and state of research on fundamental themes and similar projects to the one being presented. It opens with a dissertation about the evolution of generic object detection architectures, dividing the possibilities into single and two stages detectors and motivating the choice on which the thesis lays its ground: the YOLO family. The discussion continues with the introduction of Continual Learning as a concept and of the three possible approaches to the limitation of its main problem, that is Catastrophic Forgetting. The chapter continues by explaining the reason behind the choice of the Continual Learning Object Detection as field of research, highlighting the gaps and possibilities of improving the current scientific knowledge. Finally the main frameworks for Continual Learning experimentation are presented, as well as the reasons for which those were not used and it has been decided to build a different one.

4.1 Object Detection Architectures Evolution

As said in the previous chapter, in subsection 3.4.2, it is possible to identify two families of deep learning-based object detection architectures: they differ in the idea and practical realization of the underlying structure that is able to extract detections starting from an input image. This section aims to deepen the comprehension of the two paradigms and to follow the evolutions of their principal instances. It is important to understand and apply trade-offs between the employment of one or the other, in order to select one, basing the reasoning on factual evidence.

4.1.1 Two-stage Detectors

Two-stage detectors work by dividing the workflow into two different steps. Initially, they generate generic region proposals, that do not depend on a precise class of elements: the aim is to understand which areas in the input image are most likely to contain some object. In the second stage, classification and bounding-box regression are applied to the proposals, to complete the detection. Two stage detectors found success with the Region-based Convolutional Neural Networks (R-

CNNs) and achieved a ground-breaking architecture with the introduction of Faster R-CNN [35], that, instead of relying on hand-crafted region proposal algorithms, developed a Region Proposal Network (RPN) able to infer autonomously the presence of Regions of Interest (RoIs), and feeding these high-quality proposals to a detection head. These types of detectors were able to achieve very high-level precision and accuracy results, but at the cost of computationally heavier and slower inference pipelines. This has always made their employment more difficult in applications with real-time constraints.

4.1.2 Single-stage Detectors

Single-stage detectors get rid of the region proposal step by approaching object detection as a direct regression problem: bounding-boxes and probabilities of detection are computed at the same time in one single forward pass of the network. This approach has been introduced by Redmon et al. [34] with the YOLO (You Only Look Once) architecture. The removal of the proposal step makes this model much lighter and faster, allowing for the real-time constraints applications. For the same reason, it is however less accurate, especially on smaller targets or in dense scenarios. This gap has however been reduced with more recent versions.

4.1.3 Comparison

The main evaluation point between single and two-stage detectors is the trade-off between accuracy and efficiency. Two-stage systems aim to produce very precise solutions, leveraging their region proposal mechanism that allows this, at the price of a higher computational cost. Single-stage ones have been developed to be better suited to systems that require training and inference speed, even sacrificing accuracy, especially on very small objects or noisy images. Table 4.1 shows a summary of the comparison between the two architectures. In the context of the work that is being explained in this thesis, the basic requirements are that the platform has resource-constrained hardware and the final result has to work in real-time, so the natural choice was to prefer the single-stage detectors, specifically one of the YOLO family. This last choice will be discussed in detail in the next section.

Aspect	Two-stage	Single-stage
Pipeline	Proposal + Classification	Single forward pass
Models	Faster/Mask R-CNN	YOLO, SSD, RetinaNet
Speed	Low	High (real-time)
Accuracy	High (esp. small objects)	Good (gap is closing)
Cost	High	Low
Edge/Embedded	Limited	Highly suitable

Table 4.1: *Comparison of two-stage and single-stage object detectors*

4.2 The YOLO Family

In the landscape of single-stage detectors, the YOLO (You Only Look Once) group of models has become the de facto standard for object detection especially in contexts with real-time or real-world constraints, thanks to their focus on inference speed and accuracy, competitive even with two-stage detectors and the ecosystem that has been developed around them. It is possible to approach these models in their off-the-shelf architecture, that also comes pre-trained on generic, but very large datasets that guarantee the ability to detect common objects without requiring any specific notion about the field of computer vision. Since their original creation [34], the basic architecture has been subject of a lot of research, modifications and updates, and each of them has contributed to refine and improve the final results, always keeping alive the basic idea of a single forward pass of the network while computing bounding boxes and classifications.

4.2.1 Evolution of the Architecture

Early versions of YOLO laid the foundations for the entire family, by introducing some of the core basic concepts.

YOLOv2 [32] introduced anchor boxes, a set of selected boxes of different shapes and sizes that are regressed on the feature maps, computed by the network. This idea is useful because it makes training more stable and faster.

YOLOv3 [33] added the concept of multi-scale prediction, that is the production of detections

from different feature maps to improve the final result.

The following versions were implemented to steadily improve and work towards better accuracy and efficiency both. An important and fundamental shift has been represented with the introduction of the anchor-free version of the architecture, inside the YOLOv8 iteration [16]. This development was studied to remove the requirement of relying on hand-made and predefined anchors: in this version, the model predicts box parameters directly from the feature map. This approach made training, inference and generalization simpler.

Newer versions have been developed with a fully end-to-end inference approach in mind. The last iteration, YOLO26, is finding great appeal in the community for its low inference latency and its simple deployment pipeline. This is possible thanks to the possibility of disabiliting the Non-Maximum Suppression mechanism: this is a post-processing step that, given many possible candidates as bounding box for a single object, filters out the less probable ones, to obtain the final detection. With YOLO26, the architecture has been modified in order for it to output a single box per object.

4.2.2 The Chosen Model: YOLO26

In the family of YOLO models, the basis of this thesis' work has been chosen to be YOLO26, in its nano form [15]. This is because of the previously discussed constraints about the environment and platform that this project needs to be tested and run on: this scenario is pseudo real-time perception on an edge device, with resource-constrained hardware.

Between all the YOLO models, YOLO26 markets itself to be very efficient in deployment, real-time oriented and generally lighter and thought to be end-to-end focused in inference. It accomplishes these achievements, by implementing a lighter detection head, introducing task-specific heads for every major computer vision application and removing the necessity of applying NMS to the inference step. It is proven to have better mAP50-95 results than all its predecessor on a lot of application, including detection, which is fundamental for this thesis' sake. In addition, the nano version has been chosen between all the other possibilities, namely small, medium, large and extra. The reasoning behind this choice is self-explaining, but facts-oriented: this instance of YOLO26 is able to give both the cheapest and the fastest configuration of the model, at the obvious price of losing some accuracy with respect to the other scales. This trade-off has been evaluated to

be accepted after fully evaluating the hardware constraints.

For the aforementioned reasons, YOLO26n is the structural basic architecture on top of which the continual learning pipeline has been constructed. It has been decided to start from this stepping stone because, as highlighted in the following paragraphs, the main gap in the research is not so much the architecture as the actual methodology and implementation of a Continual Learning Object Detection framework.

4.3 Continual Learning Strategies: A Comparative Overview

From the conception of Continual Learning, a broad range of strategies has been proposed to limit catastrophic forgetting and to balance the trade-off between plasticity and stability. Following the taxonomy of De Lange et al. [5], which has become a standard reference in the field, these strategies are commonly organised into three main families: regularization-based, replay-based, and parameter isolation methods. More recent surveys further refine this picture with additional categories, such as optimisation-based and representation-based approaches [42], but the three original families remain the most established way to frame the field, and are the ones most relevant to the present work.

4.3.1 Regularization-based Methods

These methods aim to limit forgetting by adding a penalty term to the loss function that discourages the model from altering what it has already learned, without storing any past data. These methods consist of the following two approaches:

- **Weight-regularization**: based on the previous tasks, it evaluates the importance of the single parameters and computes a penalty for changes to the most critical ones; Elastic Weight Consolidation (EWC) [17], that is based on the Fisher information matrix to understand the importance, is the best-known example of this approach.
- **Function-regularization**: this penalizes the new model when the outputs deviate from those of the previous model, that acts as a teacher through knowledge distillation. Learning without Forgetting (LwF) [19], represents the principal instance of this, and uses the predictions of

the old model as soft targets for the new data.

Not requiring storage of data, these methods are memory-efficient, but they tend to grow less effective as the discrepancy between successive tasks becomes more and more vast.

4.3.2 Replay-based Methods

These methods, also called rehearsal, execute a normal training session on new data, but limit forgetting by adding and shuffling to the new samples a subset of the old ones, therefore effectively reminding periodically the past knowledge to the model that is focused in learning something new. This concept is clearly shown in iCaRL [31], that both trains with limited memory and penalizes with a loss function that implements knowledge distillation. If storing data is not possible or not permissible, we can approach the situation in two ways: generative replay replaces the subset of old knowledge with synthetically generated samples that are similar to old ones; feature replay, instead, stores intermediate representations rather than full samples, that are less memory-heavy. Replay is one of the most effective strategies against forgetting, at the cost of additional memory and of the implications of retaining past data.

4.3.3 Parameter Isolation Methods

The idea behind this class of approaches is to maintain a subset of the model's parameters statically reserved for each task: this is achievable by masking or freezing the important selected weights in a fixed-capacity network, as in PackNet [25], or by dynamically expanding the architecture with new task-specific components. This type of approach is very effective at preventing forgetting, but necessitates strict requirements to be applicable: expanding the network increases the inference latency; in addition, masking requires to know, at inference time, the task identity, limiting their employment to the task-incremental scenario.

Between these methods, the one adopted in this thesis is experience replay, in its most essential form: a small subset of raw image samples from the old domain is added to the new domain's dataset and the model is re-trained, so that past knowledge is preserved simply by composing the training set rather than by altering the learning objective. This choice is a direct consequence of the constraints of the target system. On one hand, replay is the only major method that can be applied

without a direct modification to the detector's training procedure: the underlying training procedure is then kept a closed primitive; on the other, it is consistently reported to match or outperform the alternatives at equal memory budget [26], which makes it the most favourable compromise between effectiveness and the computational cost imposed by a real-time, edge-oriented setting.

The other illustrated methods have been evaluated, but then discarded, for objective reasons, mainly regarding the time and computing budget, rather than storage memory. Weight regularization, like EWC, would add a per-step penalty learned from the old domain. However, they are known to be problematic when applied to object detection: Liu et al. [20] showed that a naive application of EWC to a detector becomes less effective if it is met by unstable optimisation and by the mismatch caused by the missing annotations of old-class objects, and requires non-trivial adaptations to become competitive. In addition, in the case of continual detection, regularization-based approaches are reported to be less performant than replay-based ones [26]. Function-regularization methods, like LwF, require an expensive forward pass of the old model at every batch of new data: this almost doubles the cost per-step, which represents a clear drawback of the approach, in particular when considering the non-trivial amount of work needed to apply these ideas to detection. Parameter isolation, as said previously, is incompatible with the target scenario: it either grows with every new domain, implying an increase in latency at inference time, very problematic for the application in autonomous racing, or alternatively requires knowledge of the new domain in a domain-incremental continual learning system, that is not available. Lastly, the generative replay option would require a generator of track and image frames in a specified resolution: this implies a higher cost than a simple storage of real images.

4.4 Continual Learning Object Detection

The two research directions discussed so far, object detection and continual learning, have largely evolved by themselves, and their unification, commonly referred to as Continual (or Incremental) Object Detection, is still a novel field, that lacks research efforts. As already seen, the majority of continual learning research considers image classification [26], and the few works that do address detection have appeared only recently [10, 21, 6, 43]. The problem was initially formalised by Shmelkov et al. [37], who defined an incremental detection protocol in which categories are

observed in sequential phases without access to past data. Scenarios beyond the class-incremental one, such as domain-incremental detection, have started to be investigated only in the last few years.

A second, more specific gap is the one due to the underlying detection architecture. The majority of the existing continual detection literature is built on two-stage architectures, usually derived from Faster R-CNN: representative examples include the work of Shmelkov et al. [37], that showed to be fundamental in the creation of this branch of research, and, more recently, distillation-based strategies such as Elastic Response Distillation [10], as well as replay-based approaches like Augmented Box Replay [22]. Further research has moved towards transformer-based detectors derived from DETR [21].

Applying a single-stage detector, like the ones of the YOLO family, to continual object detection, has, however, remained mostly unexplored territory. Zhang et al. [43] highlighted the difficult translation of methods based on two-stage or transformer detectors to frameworks that aim to create a viable solution for real time and real world applications. In this type of setting, two-stage continual object detection suffers significant drops in remembering knowledge that has previously been learned. Only a few very recent works have started to tackle continual learning specifically for YOLO-based detectors [6, 43], which confirms both the practical relevance and the current immaturity of this specific setting.

In addition, Continual Learning Object Detection is almost always studied on classical, well known benchmarks such as PASCAL VOC and MS COCO, divided in sequences of tasks. This allows reproducible and comparable evaluations, but leaves the performances of these methods on novel, custom-made datasets largely undocumented. This type of dataset is less recommended to study the behavior of a new model, because it's preferred to have shared benchmarks that allow a fair comparison of performance between different approaches and models. Constraints on data that are met in the deployment of real-world automation and robotics, in the landscape of current literature, are rarely considered.

This work, however, doesn't want to position itself in improving end results and accuracy, but to explore the world of real-time incremental detection and its possible deployment on resource-constrained edge devices. This thesis aims also to cover the gap previously highlighted, that is the overlap between lack of research on Continual Learning Object Detection and specifically of its single-stage detector variations, more suitable for real-time applications.

4.5 Existing CL Frameworks

The development of a custom framework naturally raises the question of why an existing continual learning library was not adopted instead. Over the past years, the research community has produced several well-designed tools for continual learning experimentation. Avalanche [23], is currently the most comprehensive of them as it provides support for dynamic architectures, dataset streams, and incremental training and evaluation, together with a large collection of predefined benchmarks and strategies. Continuum [8] focuses on the clean management of complex continual learning scenarios and dataset loading, while Sequoia [27] aims to unify research at the intersection of continual, reinforcement, and self-supervised learning. These tools are valuable contributions that simplify reproducible experimentation, and their utilization is well justified in many research settings.

Their design, however, is not well suited to the specific use case of this work, for three main reasons. First, these frameworks are built almost entirely around the image classification settings they do not natively keep track of bounding-box supervision, missing annotation problem, or specific metrics like mAP. Second, they are optimised for academic benchmarks rather than for the integration of custom, field-collected data coming from a real application domain, as is this case. While standard benchmarks are essential for fair and reproducible comparison, they do not reflect the data pipeline required by an autonomous system. Third, and most concretely, these libraries impose their own training paradigm, which is incompatible with the choice of building directly upon the Ultralytics YOLO ecosystem and of keeping its training procedure as a closed primitive. Adapting a real-time, single-stage detector such as YOLO26 to their internal training loops would have required substantial re-engineering, working against the efficiency and deployability constraints that motivate this thesis.

For these reasons, rather than forcing an existing classification-oriented tool onto a detection problem it was not designed for, a dedicated framework was developed, tailored to continual object detection on a single-stage detector and to the integration of custom data within a real-world, edge-oriented pipeline.

5. Contributions

5.1 Overview

As seen in the previous chapters, the gap that the following thesis' work aims to fill is understandable as the intersection between the scarcity of research about Continual Learning Object Detection (CLOD) and testing about its deployment on real-world applications with constraints that can range from limited hardware resources to real-time and safety-relevant applications. This last problem requires to execute a trade-off between the ability of respecting said limitations and good enough accuracy in inference that the deployed model represents a viable solution to a CLOD application.

The developed work has contributed to the scientific landscape on two separate, but equally important, branches, that will be presented in the following sections.

5.2 Custom Dataset

In almost every application of the generic fields of Machine or Deep Learning, one of the first and fundamental steps is the data gathering, in order to create a dataset. From these basic resources it is possible to evaluate and test various networks and architectures.

In the following paragraphs it will be discussed firstly the methodology, that is the conceptual approach to the work, and finally the implementation, that is the actual realization of a custom dataset, along with the motivations for which it has been required.

5.2.1 Methodology

The chosen context for this specific work has been the F1Tenth platform and in particular CLOD applied to the recognition and localization of opponents' race cars in the track while competing. It should be clear that the dataset required to start the training step of the project, should have been composed of a list of labeled images of RoboRacer (F1TENTH) vehicles, preferably from inside the racing track. The labels, while using a YOLO family model have to be structured each one as a `.txt` file with five numbers: `(class_id, x_center, y_center, box_width, box_height)`.

After a thorough search on the internet and on the main platforms for ML applications, only RoboFlow has been found to be useful, because it contained a correct dataset. This one has been integrated into the workflow, and it is composed of 951 distinct images, already divided in the classical DL splits, known as training, validation and testing. Most of the images are inside the training split and inside this every original image is augmented into three modified copies.

This dataset proved itself to not be nearly enough to train a successful OD model, for its lack of sheer number of samples and of negative samples, that are very useful in training. So the problem of not being able to find an already made dataset for this precise context was still standing: this work could not even start without a great effort in the creation of a custom dataset.

The natural consequence of these considerations was to create a new ad hoc dataset for the RoboRacer application. The starting point has been to reflect and decide the approach to be followed: incrementability. While trying to build a dataset that doesn't already exist, it is important to know that the current version will not be the final one and also that it can not be built by the effort of a single person. Stemming from this considerations, it is necessary to construct a system that is able to receive new data, incorporating it into the current version in order to improve and expand it.

The dataset I have created for the this work uses data that comes primarily from the on-board camera on top of the vehicle, that is a Intel RealSense Depth Camera D435i.



Figure 5.1: *Example of a RealSense Depth Camera D435i.*

5.2.2 Implementation

In a great variety of applications, ranging from robotics to automation to finance and many others, the vast majority of new data is available in sessions, during which, a stream of data is recorded.

In the use case of RoboRacer, the platform that I used as base for my work, I needed to retrieve the stream of data coming from the camera. The autonomous guide stack, developed by the HiPeRT team, leverages ROS2 (Robot Operating System) as middleware to pass information through the on board systems. The main recording tool that can be employed is the `ROS bag`. These are real-time on-the-fly recordings of the publishing topics that are running while the stack is running. While a lot of these ROS bags were already recorded from previous years, almost none of those had the camera sensor publishing data, and the few that had it were added, composing a little percentage of what has been created.

It is now clear that the majority of the data, that is, at the time of the writing of this thesis, composed of 16607 images of which 6718 are image with a detection to report, has been gathered and generated by me and the invaluable help of the team.

This was made possible by collecting the in-track recordings during sessions into ROS bags. After storing these as MCAP files, a dedicated script converts these into videos. It locates the camera topic within the bag, decodes each raw message into a standard image, and writes the resulting frames as an MP4 video. The script extract all the available frames, but the camera records images at a frequency in the order of 30 fps (frame per second).

After acquiring the video, another script slices said MP4 video into images. These images are not at the same sampling frequency of the video. This last one is 30 fps, but I have determined that keeping this high of a frequency would have inflated the number of samples, but also created useless duplicates of basically the same thing, that would not have contributed to a better end result. So I have sampled images at around 10 fps, to maintain similarity and continuity between consecutive ones, without being too repetitive.

This whole chain of MCAP $\longrightarrow$ MP4@30fps $\longrightarrow$ images@10fps, could be seen as an extra step, because it would be possible to extract images directly from a bag, but, following the idea of incrementability, and given the possibility of being given an MP4 recording of Roboracer vehicles from other sources, I have decided that creating a workflow in which is possible to inserting at every point would be more important than the efficiency in my work.

Everything described up until now is however the data gathering step. In order for the composed dataset to be usable in the following stages, such as the training of the model, it requires to be labeled. It has been said before what these labels are with respect to each image, but a tool to

create them is needed. For this purpose, it has been decided to employ CVAT (Computer Vision Annotation Tool), that is a platform created precisely for annotations for the principal applications of Computer Vision, like bounding boxes for OD, but also semantic segmentation of images and keypoints and skeleton drawing for pose estimation. This tool allows for a local deployment of itself: it is then possible to load the images that require labeling. In addition, if there is an already trained model, it is possible to automatically annotate said images, by running inference of it. From CVAT it is possible to extract labels, downloading them choosing from a format from a vast array possibilities: I clearly chose the Ultralytics YOLO for Object Detection.

All this list of operations can give us two types of output, depending on how data is composed:

- Monolithic Structure: all the images are in the same folder, so that it can be considered a single session. The dataset will be divided into the `images/` folder and the `labels/` folder.
- Distributed Structure: the starting point is divided in multiple folders one for each session, of sampling. Every folder will be composed of the `images/` folder and the `labels/` folder.

5.2.3 Dataset Composition

It has been explained why and how the custom dataset has been created for this thesis' work. In this section, it will be highlighted how the created body of data is actually composed, focusing on the reasons behind the choices. The dataset composition is as seen in Table 5.1:

Every entry of the table represents a different acquisition environment. The heterogeneity of sources is intentional: changing from indoor to outdoor, modifying the lighting, the point of view, noise of the images and their background, is useful to compose a set of images that help to make the training as effective as possible.

One of the clearest features of the composed body of data is the number of negative images: these represent almost 60% of all images. It can be counterintuitive, but images without any bounding boxes can and are even incredibly useful in teaching a model to understand to detect any object. The reason is because positive images increase the model's ability to find something, but without any negative ones it may overfit on said object, believing that every image has something to detect. The balancing of positive and negative samples is key when trying to build a useful deep learning training mechanism. The presence of negative images allows to also compute the FAR (False Alarm

Dataset	Images	Positive	Negative
f1tenth.v2i.yolov11	2283	2283	0
huawei-uci2020-dataset	514	60	454
squeaky_17-04-26	3270	850	2420
objects_negatives_15-05-26	1165	0	1165
FIM_outside	3378	1932	1446
FIM_inside	2441	894	1547
icra_test_camera	3556	699	2857
Total	**16607**	**6718**	**9889**

Table 5.1: *Composition of the collective dataset.*

Rate), that is a useful metric in a safety-critical environment, avoiding false detections that could disrupt the vehicle ability to work correctly.

5.3 Continual Learning Framework

5.3.1 Overview

It has been shown all the efforts made towards the creation of a custom made dataset, but it has not been work as an end in itself, but it was needed as a stepping stone towards the implementation of a framework that allows the creation of experiments to study the effects and possibilities given by the usage of the Continual Learning paradigm. This framework has been built in two main parts: Preprocessing and Benchmarking.

In the Preprocessing section, every step has been approached trying to guarantee incrementability and transparency to the end user, by making it as simple to interface with the work as possible. The idea is to be able to add new batches to the general dataset with as little effort as possible, with a single access point, if some general guidelines are respected. The same could be said for the addition of new experiments, by adding an entry to a configuration file. All these processes could have been abstracted another step, by further abstracting from the project itself, but time constraints

made this idea unachievable.

In the Benchmarking stage, the idea was to separate the computationally heavy steps from the cheap ones. This objective was solved with the purpose of being able to run just once the complex and slow part of benchmarking, possibly on accelerated hardware like GPU, and, if needed, running the cheap section multiple times, modifying metrics, thresholds or generally anything needed. This can be achieved by writing in memory the results of the first part of the benchmarking and once we have these recorded, it is pretty fast to complete everything else.

5.3.2 Preprocessing

The preprocessing step has been created aiming for a way to load the single sessions that compose the base of images in the pipeline. This has been achieved by defining two classes, that represent abstractions of the concepts of image and dataset:

```
@dataclass
class ImageSample:
    status: LblStatus               # status can be ("positive", "negative")
    ds_name: str                    # name of the root directory that contains the session
    img_path: Path                  # absolute path to find the image
    lbl_path: Path | None = None    # absolute path to find the corresponding label (can
                                    # be not existent)
    split: Split | None = None      # links the sample to its designed split for training
                                    # (can be not existent)
    group: str | None = None        # name of the precise sub-session of recording, that
                                    # is the first level under ds_name (can be not existent)
```

Code 5.1: *ImageSample dataclass definition*

```
@dataclass
class Dataset:
    name: str                       # name of the root directory that contain the session
    root: Path                      # absolute path to find the session
    pos_samples: list[ImageSample]  # list of the positive images of the session
    neg_samples: list[ImageSample]  # list of the negative images of the session
```

Code 5.2: *Dataset dataclass definition*

These two abstractions allow the loading of every session of sampling of which the dataset is composed of. This is however not enough, because it is important to create a way to automate said loading. This was achieved by the definition of a base loader interface, that was instantiated in two

different ways: the first allows to load sessions saved in a monolithic structure, while the second is applicable to dataset with a distributed structure.

The interface asks to implement a class with an initialization method and a method that searches samples from a root path:

```
class Loader(ABC):
    @abstractmethod
    def __init__(self, root: Path):
        self.root = root    # it is required to give as argument to create a
                            # loader at least the root of the session

    @abstractmethod
    def find_samples(self): # method to find and load samples
        ...
```

Code 5.3: *Basic Loader Interface*

The two loaders that have been actually implemented in my framework are `StandardLoader` and `SubfoldersLoader`. The two differ mainly on the fact that the first can access directly to the given root directory, while the second has to iterate over every subfolder contained in the root. After accessing to the correct folders, the logic is however the same:

```
for img in sorted(img_path.iterdir()):  # iteration over all the images
    if not img.is_file():               # safety control
        continue

    file_name = img.stem                # saving the name of the image (no extension)

    lbl = lbl_path / f"{file_name}.txt" # building the label path as combination
                                        # of the absolute path of the labels root
                                        # and the image name followed by the .txt
                                        # extension

    if not lbl.is_file():
        neg_samples.append(ImageSample(
            status = LblStatus.NEGATIVE,    # if there is no label, save the image
                                            # as a negative
            ds_name = self.root.name,
            img_path = img,
            group = self.root.name,
        ))

    elif lbl.stat().st_size == 0:
        neg_samples.append(ImageSample(     # if the label is an empty file, save the
```

```
                                        # image as a negative
            status = LblStatus.NEGATIVE,
            ds_name = self.root.name,
            img_path = img,
            lbl_path = lbl,
            group = self.root.name,
        ))
    else:
        pos_samples.append(ImageSample(
            status = LblStatus.POSITIVE,    # if the label is a file with indications
                                            # for a bounding box, save the image as a
                                            # positive
            ds_name = self.root.name,
            img_path = img,
            lbl_path = lbl,
            group = self.root.name,
        ))

return pos_samples, neg_samples # it returns all the samples divided as positive and negatives
```

Code 5.4: *Image loading logic*

All this has been created with the purpose of easing the introduction of differently-formed datasets into the central one: if another user aims to do so, there are just a few easy steps that are required. Firstly, the implementation of a custom loader with the logic required to navigate its data and returning correctly positive and negative samples as distinct lists of ImageSamples data. The process to insert a new dataset then requires to open the `datasets.yaml` file and adding a new entry with the following structure:

```
datasets:
  dataset1:
    name: "f1tenth.v2i.yolov11"
    root: "/home/path/to/project/datasets/f1tenth.v2i.yolov11"
    type: "subfolders"  # or "standard"
```

Code 5.5: *datasets.yaml generic entries*

The 'type' specific is a direct indication of which loader has to be employed to complete the addition of a new dataset. The logic of this is:

```
def load_datasets():
    datasets = []

    ds_infos = dyp.get_ds_infos()
```

```python

    for ds in ds_infos:
        if ds['type'] == 'standard':
            loader = StandardLoader(Path(ds['root']))
            pos_samples, neg_samples = loader.find_samples()
            datasets.append(Dataset(
                name = ds['name'],
                root = ds['root'],
                pos_samples = pos_samples,
                neg_samples = neg_samples
            ))
        elif ds['type'] == 'subfolders':
            loader = SubfoldersLoader(Path(ds['root']))
            pos_samples, neg_samples = loader.find_samples()
            datasets.append(Dataset(
                name = ds['name'],
                root = ds['root'],
                pos_samples = pos_samples,
                neg_samples = neg_samples
            ))
        else:
            # For differently-composed datasets it is needed to add to
            # this if ... else ... other conditions
            raise ValueError(f"Unknown dataset type: {ds['type']}")

    return datasets
```

Code 5.6: *Centralized addition of datasets*

The approach is summarizable in the diagram in Figure 5.2.

As said before, the framework's main purpose is to conduct experiments given a composable dataset. To create new ones, the framework requires to just add a new entry to the `experiments.yaml` file that has the following structure:

```yaml
experiments:
  exp1:
    name: "base_90_10_s1"   # name of the experiment
    model: 'yolo26n.pt'     # base model to start from
    arm: "base"             # analytical label
    tag: "90_10_s1"         # same as base

    # every experiment can change some wanted hyperparameters from a
    # standard set, that can be found in training_supervised/train.py
    hyp: {epochs: 100, patience: 100, seed: 1}

```

```
    # here we can select how many and which sessions to use to train
    # the selected base model, in fact composing the dataset
    # it is possible to also choose the percentage of positive and
    # negative samples to control the ratio between them
    datasets:
      f1tenth.v2i.yolov11:        {pos_ratio: 1.0, neg_ratio: 0.1022}
      huawei-uci2020-dataset:     {pos_ratio: 1.0, neg_ratio: 0.1022}
      squeaky_2Macchine:          {pos_ratio: 1.0, neg_ratio: 0.1022}
      squeaky_1Macchina:          {pos_ratio: 1.0, neg_ratio: 0.1022}
      objects_negatives_15-05-26: {pos_ratio: 1.0, neg_ratio: 0.1022}
      FIM_outside_base:           {pos_ratio: 1.0, neg_ratio: 0.1022}
    val_pct: 0.15           # percentage of images in the validation set
    seed: 42                # seed to initialize a random function
```

Code 5.7: *experiments.yaml generic entries*

The framework is then able to randomly mix and write the paths to the images from the given datasets to three files per experiment: `train.txt` and `val.txt` inside the `'experiment_name'/` folder, and an additional file, `config.yaml` that is composed of indications in a YOLO standardized format that lets the model know where training data can be found. Because the dataset contains temporally adjacent video frames, this procedure may place highly correlated frames in both partitions and may therefore overestimate validation performance. A session-level or temporally blocked split is required in future evaluations.

After all these operations, the data is then able to be used for the training itself. This step has been approached in the standard way Ultralytics YOLO gives to fine tune their previously trained models, modified by iterating it over all the created experiments, and for each, giving the possibilities to modify some or all the hyperparameters to adapt to the purpose of the precise iteration. Some default parameters have been selected a priori:

```
TRAIN_DEFAULTS = dict(
    # --- core ---
    epochs = 100,
    patience = 20,          # early stop if mAP doesn't improve for 20 epochs
    imgsz = 640,            # match the on-vehicle inference resolution
    batch = 16,             # fixed value: -1 auto-picks from free VRAM

    # --- optimizer ---
    optimizer = 'AdamW',    # more stable convergence than SGD for fine-tuning
    lr0 = 0.001,            # below default 0.01: a lr too high would hurt since we're fine tuning
    lrf = 0.01,             # final lr = lr0 * lrf = 0.00001
    warmup_epochs = 3,      # keeps lr higher for the first epochs
```

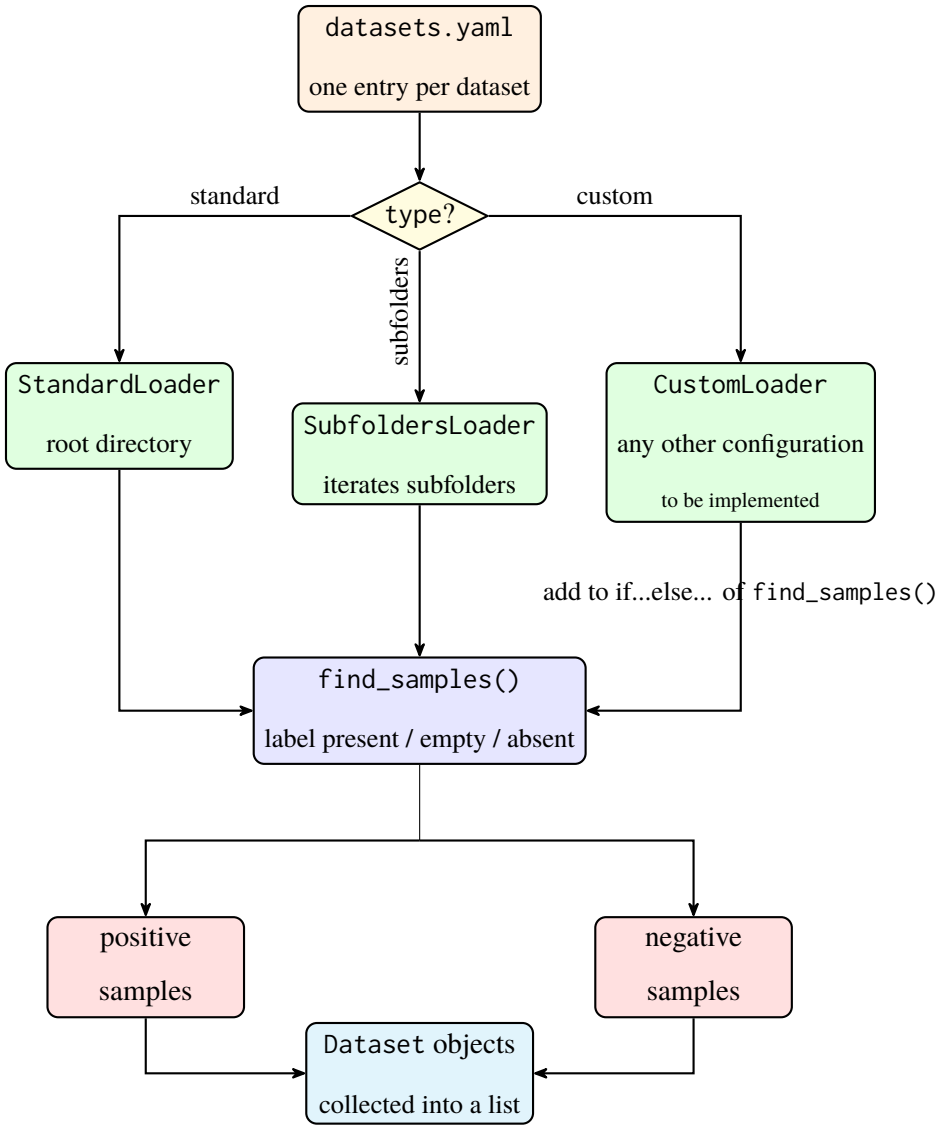


Figure 5.2: *Logical flow of the dataset loading process.*

```
    weight_decay = 0.0005,  # regularization to limit overfitting

    # --- loss weights ---
    cls = 0.5,              # with a single class this value is the confidence itself

    # --- augmentation ---
    hsv_h = 0.015,          # hue jitter, kept small as lighting is usually consistent
    hsv_s = 0.5,            # saturation jitter
    hsv_v = 0.5,            # brightness jitters (handles shadow variation)
    fliplr = 0.5,           # horizontal flip (opponent can come from either side)
    flipud = 0.0,           # vertical flip (disable: cars don't appear upside down)
    mosaic = 1.0,           # simulates multiple opponents/distances
    mixup = 0.0,            # off: it can create semi-transparent cars (false negatives)
    scale = 0.5,            # scale jitter (simulates distance variation)
    translate = 0.1,        # position jitter (handles non centered opponent)
    degrees = 5.0,          # small rotation (camera is roughly fixed on car)
    perspective = 0.0001,   # subtle perspective warp for robustness

    # --- evaluation ---
    iou = 0.7,              # NMS IoU, already described

    # --- hardware ---
    device = 0,             # use GPU device
    workers = 8,            # cores number

    # --- reproducibility ---
    seed = 42,              # fixed RNG seed
    exist_ok = False,       # raise error instead of overwiting an existing run
)
```

Code 5.8: *Default hyperparameters for training*

5.3.3 Benchmarking

The previous section represents the description of the preprocessing part of the work, and the training itself, iterated over the experiments. The test step is however nowhere to be seen, because it has been decided to separate it from training and validation, to better control against which images and samples the performance of the trained models would have been evaluated. The benchmarking stage concerns itself of all these considerations.

As far as the management of the data is concerned, almost the same structure created for the dataset handling has been employed: the final user is asked to interact with a `benchmarks.yaml`

file, stating the same parameters for every group of data, that is then loaded onto the actual pipeline.

```
benchmarks:
  benchmark1:
    name: "icra_holdout"
    root: "/home/path/to/project/datasets/icra_test_camera/icra_holdout"
    type: "subfolders"
    enabled: true
```

Code 5.9: *benchmarks.yaml generic entry*

As it is observable, there is only an additional parameter, `enabled`, that just indicates if the precise benchmark is wanted to perform testing on.

In this section the guideline was to separate the computationally heavy parts to the cheap ones. As stated before, the purpose of this approach is to compute a single time the slow inference step once, while making the computationally light operations modifiable and repeatable as many times as needed. This is feasible by running said inference a single time for every pair of model and benchmark and subsequently writing in memory information of data and the linked predictions the model made about it. These records are saved as a JSON record and put together in a JSONL file in a `dump`, in the following way:

```
{
    # path to the predicted image
    "image": "/home/path/to/project/datasets/FIM_inside/test_1_03_09_2026/images/frame_000.jpg",

    "group": "test_1_03_09_2026",   # group the image belongs to
    "status": "positive",           # labeled or not
    "w": 1280,                      # horizontal dimension of the image
    "h": 720,                       # vertical dimension of the image

    # predicted box values (x1, y1, x2, y2) with confidence score
    "pred": [{"box": [690.61, 89.68, 1271.66, 600.39], "score": 0.86274}],

    # real box values
    "gt": [{"box": [688.54, 73.01, 1265.64, 602.17]}]
}
```

Code 5.10: *example of dumps of predicted detections*

Together with the dump, a JSON file is produced, containing metadata that describes under what conditions said dump has been produced:

```
{
```

```
  "meta_version": 2,                    # format version of this file

  # weights used for predictions and its SHA-256 hash
  "weights": "/home/tmortara/research/utils/training_supervised/results/base_50_50_s1/weights/best
    .pt",
  "weights_sha": "2fb958c05764a65c83a82c717e12b7251e04c4477870215336c5ebed57dd3dc0",

  "benchmark": "FIM_inside",            # which benchmarks have been predicted
  "n_images": 2441,                     # how many images have been predicted

  # next three are inference parameters
  "conf": 0.001,
  "iou": 0.7,
  "imgsz": 640,

  # machine used to run inference, along with specifics
  "host": "forrester",
  "device": "NVIDIA GeForce RTX 5060",
  "torch": "2.14.0+cu130",

  # time results
  "infer_ms_per_img": 2.847,
  "wall_ms_per_img": 5.183,
  "wall_s": 12.65
}
```

Code 5.11: *example of dump's metadata*

The dumps contain the concrete results of prediction, while the metadata files serve precise purposes and all can be reconducted to caching and storing information about the inferences themselves.

By storing information about a precise dump, most of all the SHA-256 field, it is possible to avoid the risk of executing runs of inference more than once: this is usually not wanted as it represents a long and expensive task that does not produce very different, or different at all, results. The hash field is important because every model's weights file, even if with the same name, produces a different result after its hashing, avoiding the possibility of producing same results because of forgetfulness or errors. The `meta_version` field is there to explicitly run inference again, if wanted or needed.

Other fields, more notably the machine's name and device, have been introduced in order to make sure that the comparison between models and inferences would make sense: to relate and

discuss differences between results of models on different machines is not informative on many useful metrics.

It is important to highlight the fact that the field of `conf` is 0.001: every prediction with a higher confidence score than the specified one, so almost everything, goes on to be stored in memory. This allows for new analysis and questions about models, to be able to be created and new answers to be computed at the price of just seconds of CPU, by just applying new slices and modifying the operating threshold. This very low confidence value is also necessary in order to correctly retain all the necessary information to trace the precision-recall curve, fundamental to compute the Average Precision metric, which will be used later. In later stages the confidence threshold will be set at 0.25, that represents the minimum confidence that a prediction has to have to be considered in the computing of the requiring metrics, and, specifically for this use case, the False Alarm Rate (FAR).

After this, slices are computed. Slices are division of data by one of their own characteristics: they can be grouped if they are positive or negative, if the object covers a small, medium or large portion of the entire image area or if they belong to the same group of origin. These slices can be very useful to confront particular conglomerates of data, and making considerations about them. It is important to note that in order to modify, add or delete new slices, it is necessary to act on the following function, that doesn't require new inferences, because it is just a matter of filtering on data that is already available on disk.

Raw data from dumps, split into subsets, logically divided in slices, are then grouped and written in a new file, called `ledger`. This is written once per session of training: every line is a combination of the pair (model, benchmark) and contains all the required metrics, precisely one set of them per every defined slice. The ledger file is quintessential as the line of separation between all the heavy computation that should run once, and the cheap operations, that can be run as much as needed, because the cost is very limited.

In Figure 5.3, it is presented a graphical representation of the benchmarking pipeline, for clarity's sake.

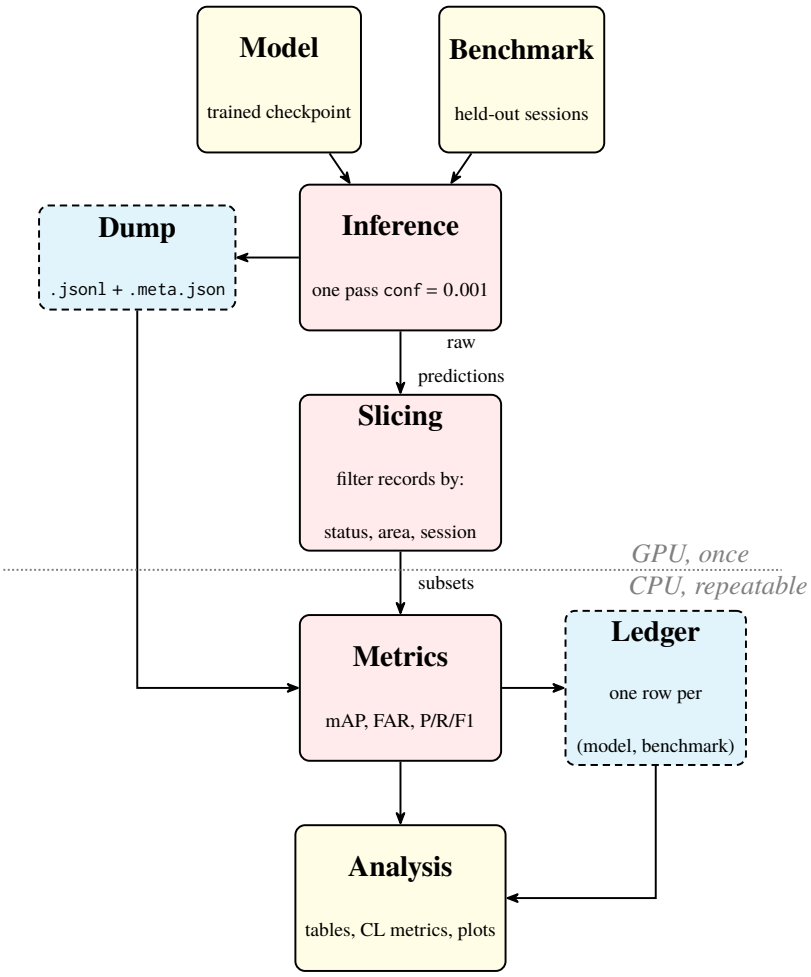


Figure 5.3: *The benchmarking pipeline.*

6. Results

6.1 Research Questions

In chapter 5, it has been explained in detail how the custom dataset and the Continual Learning framework, have been developed, both from a methodological or conceptual standpoint and from the concrete architectural and coding choices. It has been explained that the employment of this tool can allow the conduction of experiments of various nature, including, but not limited to, Continual Learning scenarios. This has been the main talking point for this work, as it has been found to be particularly interesting and well suited for possible gaps and advancements in the scientific research, particularly in the field of real-world autonomous systems application.

In this landscape and after a careful analysis of the literature to understand weak points of this type of learning, it has been decided to conduct a thorough investigation on a situation that represents the base of continual learning and of its abilities, but also of its problems. The adopted situation refers to the presence of a specialized base dataset, able to perform object detection on its own, and to study the best approach for the introduction of a new session. In this context, the employment of different and thought-out benchmarks can allow considerations of the resulting models' ability to:

- **Forget** previously learned data: this case represents the possibility of insurgence of the phenomenon of catastrophic forgetting. The models could take steps towards overfitting on the newly acquired environment, risking a drop in performance on the same task that it could complete before.
- **Generalize** on a never-seen environment: in this section the main focus is to understand if and how much the new model is able to expand its ability of completing a task when facing data that comes from an environment which it is not trained on.
- **Specialize** on a just-acquired environment: there are many applications where the ability of a system to be able to adapt effectively to a situation that has been just presented is much needed.

The aforementioned theoretical situation that has been presented as starting point for the fol-

lowing discussion is significant in this thesis, because it represents an iteration of a continuously learning device and how it should go about incorporating new data in its system.

6.2 Benchmarks and Experiments Set-Up

The research questions illustrated above represent the core of the more scientific research part of this work. Before computing a single result, one of the first critical decisions was the choice of datasets to leverage when testing the generated models. Since the research questions are three, every model has to perform actions that give an answer to each one, and the way to do so is to run inference of the model on different datasets. In the work three benchmarks have been decided:

- **FIM_outside_holdout**: this data is a set (1773 samples) that has been kept from training from the session FIM_outside, composed of 49.7% negatives. It is employed to test the final resulting model's ability to repeat the initial task on which it was trained on: by evaluating performance on this dataset it is possible to understand if the model is able to repeat the original actions it was able to perform.
- **FIM_inside**: these samples are never seen by the model in training. The dataset is composed of 2441 images, 63.4% of which are negatives. Its purpose is to help to understand if and how much a given model is able to perform the task it has learned on a brand new environment.
- **icra_holdout**: this sub-set of the `icra` dataset is composed of 886 images, 66.7% of which are negatives. Its purpose is to understand how much the new model has become able to perform the task of detection on the newly proposed environment and if it is on the same page as the other benchmarks or if it is possible to note if it is better or worse than the average.

As stated previously, the initial step that the analysis requires before starting is the development of a base model able to perform the required object detection in the specific domain of opponent's detection via camera. In order to make this result available, training has been run, starting from a YOLO26n model, pretrained on the generic, but very large, COCO dataset, and adding specialization by feeding it multiple acquired sessions. Even in this apparently straightforward section of the work, a doubt crept in: in a domain largely dominated by negatives, as it is more common to not see another racecar than to see it, is it possible to determine a "good" percentage of negative values with respect to the total of the available images?

To answer this question, it has been decided to perform four different trainings to compute a base model. The first and most simplistic idea was to employ the selected sessions as the base dataset in their entirety: this decision coincidentally implied to use a percentage of positives very near to 50% of all the samples (47.9%). The other proposed solutions all circle around the idea of increasing the percentage of positives with respect to the total images proposed, by filtering randomly negative samples: 70/30 and 90/10 splits were the precise ones that has been developed. Lastly, it has been explored a solution that proposed the employment, as negative samples, of the so-called hard ones, that contain images with no detections, but with other objects or particular backgrounds that make the image itself very noisy, but also very informative. These four approaches were trained three times each, every time with a different seed as random generator, to ensure a lack of dependency to luck in the results, as just the closest to the average of the three was used in the further considerations.

The final choice of model to employ as base for the rest of the work was the ~50/50 split, to have the best, considering mAP50-95 and FAR, initial standpoint: this is important as it is a necessary choice to simulate the working of a real environment, where it would not be chosen a sub-par model to run on a real device.

The natural next step was to decide how to approach the central objective of addition of new data to the central dataset, and the proposed ideas are three:

- **Joint Retrain**: this approach has as a central idea to start from the same starting point, the YOLO26n model, and repeat training of it using the old composable dataset to which it is added the data coming from the new environment's acquisition session. This is the most classical idea, as well as clearly the heaviest, as far as computing complexity goes, but it should intuitively represent the absolute best in terms of performance.
- **Naive Fine Tuning**: this approach has as a central idea to take the resulting model of the previous step, that is the baseline model trained on a complete dataset, and build from it a fine tuning training that leverages only the new environment's acquired data. For its simple and clear-cut way of reasoning, it has been defined "naive" and should theoretically act as a lowest bound in terms of accuracy: it mimics the Continual Learning mechanism in its unadulterated approach, that has a well known problem of developing catastrophic forgetting.
- **Fine Tuning with Experience Replay**: this approach represents one of the ways in which it

has been shown possible to mitigate the effects of the aforementioned catastrophic forgetting. It involves to feed a percentage of the data used to train the base model together with the data from the new environment in the network to ensure a limited risk of forgetting previous domains. This is a compromise between the two ideas expressed before.

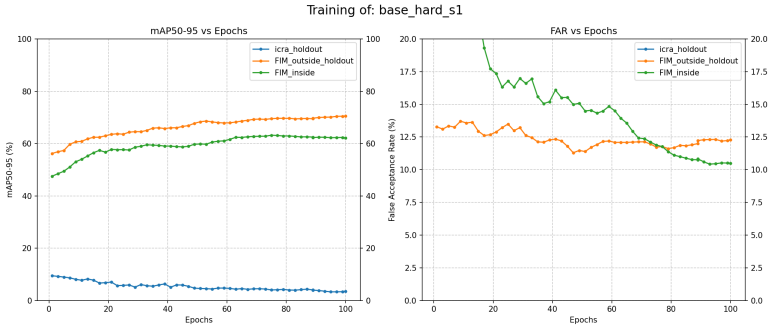
All three methods were evaluated across different fractions of the base data budget, which is the quantity that make them different. Naive fine-tuning is always zero by definition: its percentages instead vary the amount of new data (10%, 50%, 100%). Experience replay was run with buffers of 2%, 10% and 50% of the base set, and joint retraining with 2%, 10%, 50% and 100%. The three overlapping levels are deliberate: at 2%, 10% and 50% the joint and replay configurations are trained on exactly the same images and differ in whether the network is initialised from YOLO26n or from the base model, other than the corresponding set-up (epochs, learning rate and warm-up). The 100% joint configuration serves as the conventional upper bound.

In addition, it is worth noting, for completeness' sake, that not only the base models' training, but every approach has been run on three different randomness generating seeds, still to ensure to not risk the presence of lucky results, that could compromise the general discussion.

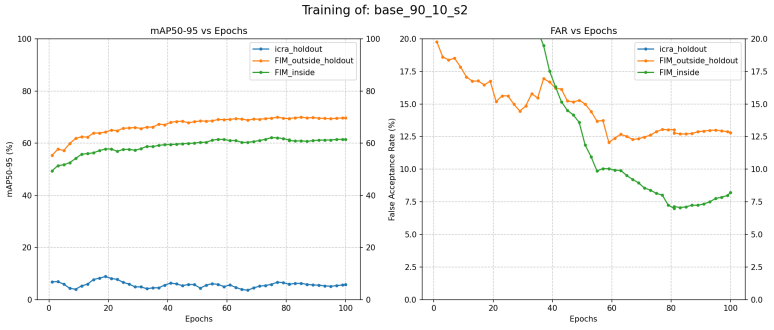
6.3 Results

In the previous paragraph it has been illustrated which experiments have been proposed, and which specifics have been deemed as important to create both variety and consistency. In this section it is possible to see the actual results of the research efforts. They will follow the order previously used in section 6.2, and will be evalutated on metrics tracking average precision (mAP50-95) and false positive generation's tendency (FAR).

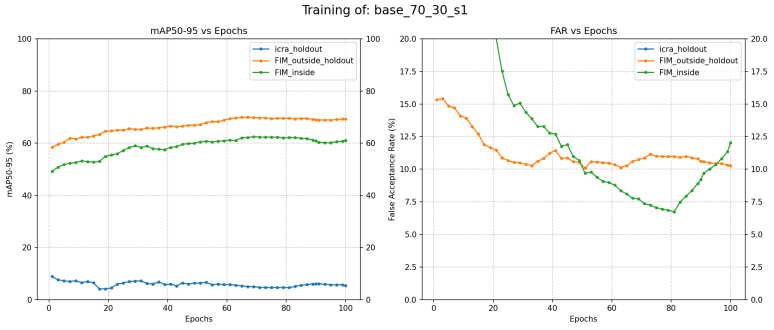
The first important step that requires analysis and result's computation was the base models' creation. It has been decided to employ ~50/50 split, that coincidentally uses all the images selected for the base dataset, but also 70/30 and 90/10 sweeps, where negative samples have been filtered randomly, and finally a run that uses only hard negatives, that are samples with no detection, but very informative as they contain noisy backgrounds or a number of object different from racecars. After running training on the various splits, the outputs are what is possible to see in Figure 6.1.



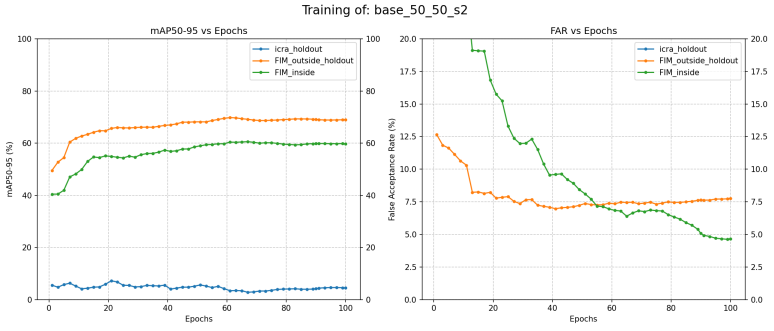
(a) base_hard_s1



(b) base_90_10_s2



(c) base_70_30_s1



(d) base_50_50_s2

Figure 6.1: *Training curves (mAP50-95 and FAR vs epochs) for the base model under different data-split configurations.*

As it is possible to read from the plots, the four configurations are indistinguishable on the average precision in detection: they differ on just the False Alarm Rate (FAR) that favors clearly the complete (~50/50) split. In the autonomous racing scenario, this metric holds great relevance as it is hurtful to falsely detect opponents, as it may cause braking under no real necessity.

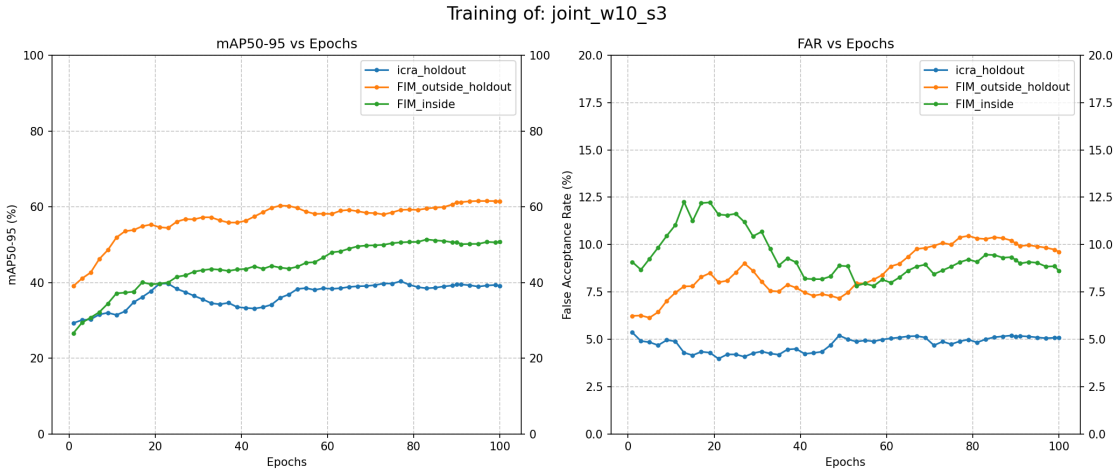
For the following demonstration, and after a selection of the runs closest to the mean of the results of multiple seeds per training, it has been decided for the illustrative picture to limit them to just one for approached method. The proposed solutions are however not selected by their performance, but by the global design they were created on. The actual selection was to include the naive fine tuning on the entire new dataset, the joint retrain on 10% of old data, starting from the generic YOLO26n model, and finally the experience replay on a buffer of old data of 10%.

This approach has the purpose to try and understand the behavior of models that differ by just one structural decision, or as close as possible to it: experience replay and naive fine tuning express the difference in being able to repeat observations on a buffer of old data, while experience replay and joint training are separated only by the choice of starting model, respectively one the computed base and one the generic YOLO. By selecting 10% of the old data, the images on which they are trained are the same and to compute differences and to compare performances is almost independent from unwanted additional confounders, leaving very informative answers.

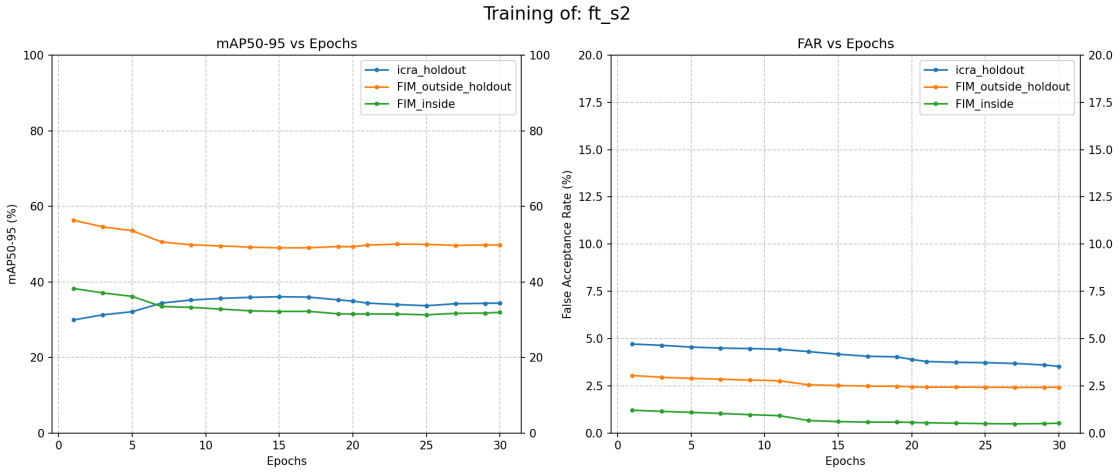
In Figure 6.2 it is possible to see the behavior of the three models in separate plots, each containing three curves, one for every benchmark we have defined, each representing one of the core research questions: specialization (blue), forgetting or recall (orange) and generalization (green).

Figure 6.3 has a similar purpose, but with a more comparative approach on the behaviors on the same benchmark, adding the joint training with 100% of all data, that represents the conventional upper bound. The different number of epochs reflect two different learning problems more than an arbitrary choice. Joint retrain configurations (blue) start from pretrained weights and must learn the detection task in full, and are therefore given 100 epochs at the standard learning rate. Fine tuning (red) and replay (green) start from an already trained model and only adapt it, at a learning rate of 10^{-4} with no warm-up, and are given 30.

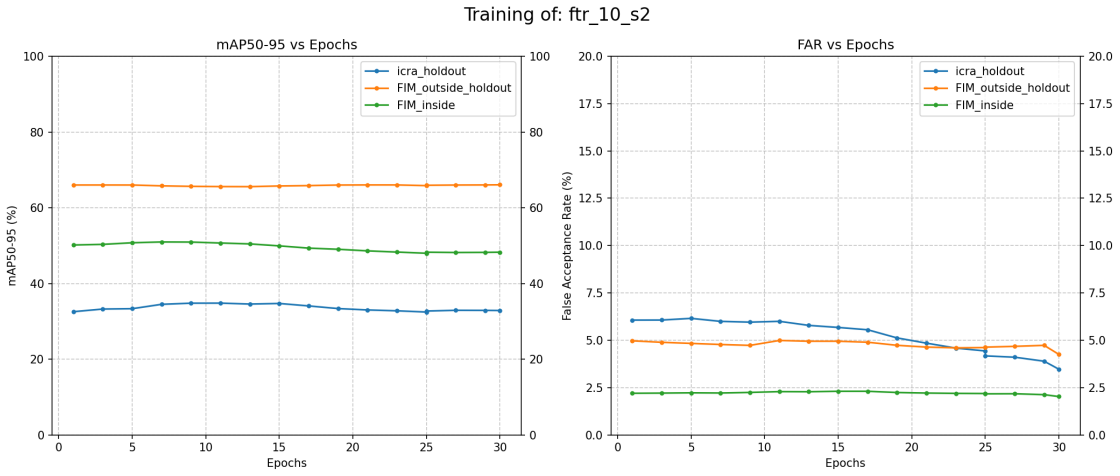
It is also important to highlight the duration of training steps per computed model: in Table 6.1 it is possible to read these with the addition of indication of total number of images and epochs



(a) *Joint retrain, 10% budget (*`joint_w10_s3`*)*

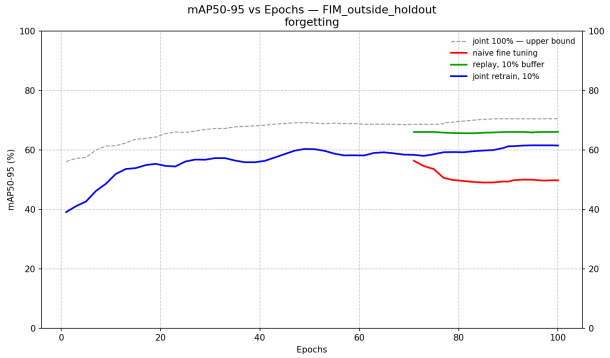


(b) *Naive fine-tuning (*`ft_s2`*)*

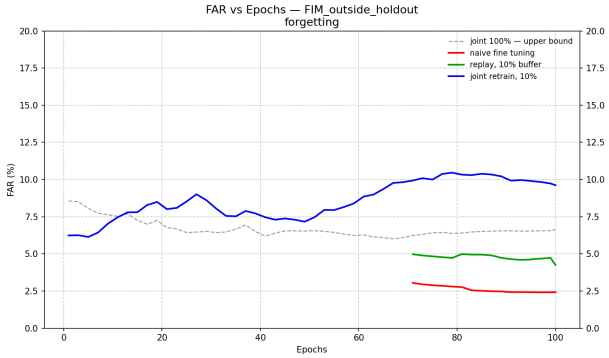


(c) *Fine-tuning with experience replay, 10% buffer (*`ftr_10_s2`*)*

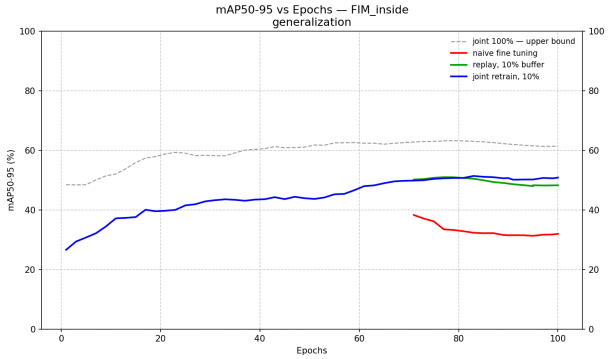
Figure 6.2: *Training curves (mAP50-95 and FAR vs epochs) for the three adaptation strategies.*



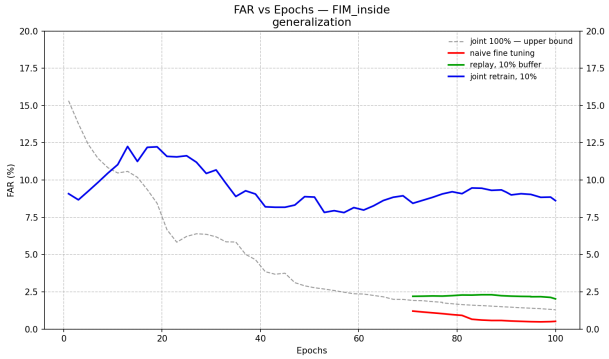
(a) mAP50-95 — `Forgetting`



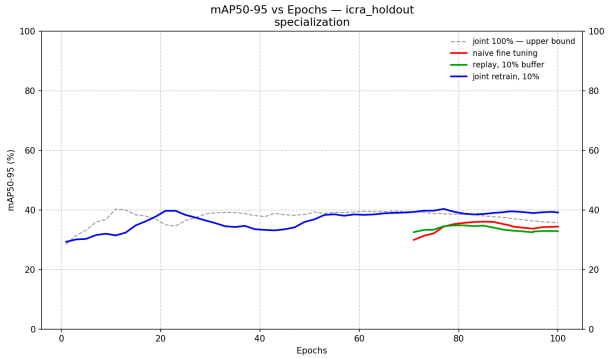
(b) FAR — `Forgetting`



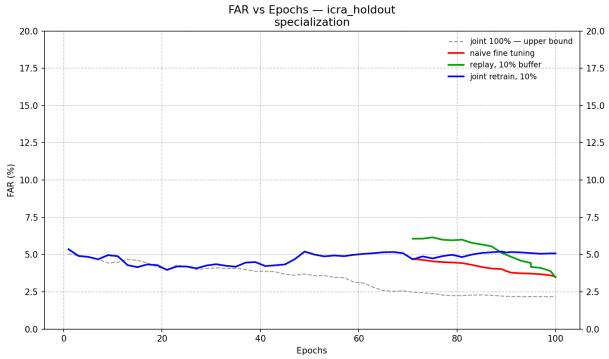
(c) mAP50-95 — `Generalization`



(d) FAR — `Generalization`



(e) mAP50-95 — `Specialization`



(f) FAR — `Specialization`

Figure 6.3: *Comparison of the three methods across the three benchmarks: mAP50-95 (left column) and FAR (right column), for* `Forgetting`, `Generalization` *and* `Specialization` *(rows).*

employed. Training processes that stem from the idea of fine tuning an existing model require less epochs because to employ too many iterations could cause the resulting weights to overfit on the new main environment.

Arm	Configuration	Images	Epochs	Minutes
Base	`90/10`	4 705	100	32.8
	`70/30`	6 048	100	41.2
	`full`	8 837	100	58.6
	`hard`	6 048	100	41.2
Joint retraining	`2%`	2 846	100	19.1
	`10%`	3 552	100	23.8
	`50%`	7 088	100	47.4
	`100%`	11 507	100	76.2
Fine-tuning	`10%`	267	30	0.7
	`50%`	1 335	30	2.8
	`100%`	2 670	30	5.4
Replay	`2%`	2 846	30	5.8
	`10%`	3 552	30	7.2
	`50%`	7 088	30	14.2

Table 6.1: *Training cost of each experimental configuration.*

6.4 Discussion

After the results gathering, it has been decided to compute a discussion of it separating arguments and conclusions by research questions, and it will be useful to refer to the Table 6.2 for ease of reading and understanding. It is important to note that the scale on which the results are classified as excellent (red) to underwhelming (green) is extracted from the literature for the mAP50-95, while is self-centered in the case of FAR.

The table is set-up to show distinctly the values of the accounted metrics (mAP50-95 and FAR) and splits the two main columns each in three, one division for each research question. Every

configuration	mAP50-95 ↑			FAR ↓		
	icra specialization	FIM_out forgetting	FIM_in generalization	icra specialization	FIM_out forgetting	FIM_in generalization
baselines						
`yolo26n` (pre-trained)	0.1%	0.6%	1.0%	91.2%	52.8%	35.4%
base model	4.6%	68.7%	61.2%	40.3%	7.8%	5.0%
joint retrain						
2 %	36.8%	53.4%	38.0%	3.6%	4.8%	4.1%
10 %	35.5%	61.8%	51.2%	2.9%	6.8%	6.1%
50 %	**38.1%**	68.3%	56.3%	2.8%	5.3%	3.7%
100 %	37.7%	69.3%	**61.3%**	**2.7%**	7.3%	2.0%
fine-tuning						
10 % data	22.6%	63.2%	48.8%	7.8%	4.9%	3.3%
50 % data	33.2%	48.2%	29.5%	4.0%	2.9%	1.0%
100 % data	35.2%	51.2%	33.1%	3.7%	**2.5%**	**0.5%**
fine-tuning + replay						
buffer 2 %	34.7%	61.4%	42.1%	3.8%	3.4%	1.1%
buffer 10 %	34.0%	66.5%	49.6%	4.6%	4.4%	2.3%
buffer 50 %	33.6%	**69.4%**	54.9%	4.5%	5.1%	2.7%

Table 6.2: *Complete results of central inquiry. 3 seeds average.*

row can be read as the performance of a single computed model, whose specifics are explained in the first cell, for every question first by its mAP50-95 and then on its own tendency to predict false positives. Every column does therefore represent a comparison of all the models on a precise research question. It is important to note that the results reported in this table are not the run closest to the mean as the previous plots, but it has been decided to insert the average values between different seeds' runs: the plots aim in fact to report performance on a single run, specifically the one closest to the middle, while in the summary table we report the actual average of the metrics for every run on every benchmark.

6.4.1 Forgetting

Regarding the research question that looks into the Forgetting or, complementarily, the ability of the model to retrieve past information and abilities it had, it is possible to highlight some predictable conclusions, but also unexpected ones.

The mAP50-95 of detection on the `FIM_outside` benchmarks lets through clearly the message that Fine Tuning (FT) has the lowest results possible, as it could be predicted, having explained in depth the possible effects of catastrophic forgetting in this approach. When employing FT, either a small percentage is leveraged or coming across some degree of forgetting about old tasks is inevitable. By feeding just a low number of new samples, it is possible to keep the knowledge retrieval pretty high, but it is a vain procedure as it does not add data and abilities to the model.

By reading the gathered data, it is possible to observe an almost equality in mAP50-95 of Fine Tuning with Replay (FTR) and Joint Retrain (JR), as the differences stay within a few percentage points. Analyzing the FAR side of the table, however, FTR is able to show a lower tendency of false positives on many of the results of the board, while being at least equal on mAP50-95.

6.4.2 Generalization

Talking about the ability of the resulting models of Generalize on never seen environments, the conversation changes.

FT remains by far the worst option in terms of mAP50-95, even if it presents astoundingly low FAR scores: to naively fine tune a model on a precise domain, undermines the model's own ability

to execute its task on a never seen environment.

The comparison between JR and FTR, however, this time prefers a more robust approach, that is a full retrain starting from YOLO26n, with a lot of epochs, using all the possible data from the base dataset and the new session. This is not a difference that could be attributed to noise or error, but a large gap of 6% that is notable. With these settings, the JR method presents very good results in terms of FAR, making it a strong and stable solution.

Noting that the FTR results are still acceptable, decent even, is a statement worth saying, for reasons that will be discussed later.

6.4.3 Specialization

Finally it is possible to speak about Specialization. This particular branch was difficult to predict and shows a surprising revelation: the percentage of mAP50-95 and of FAR are incredibly close. They are unsurprisingly lower than the other tested fields, but, apart from FT on 10% that shows clearly bad results, everything else can be found in a range of 5%, that for nine different runs is not a lot.

Another surprise is the lack of improvement or even deterioration of both metrics when varying the percentages of data fed with training.

6.4.4 Conclusions

The conclusions are unsurprisingly not clear-cut: there is no one sure and perfect solution that can be employed in every scenario and can be considered always the best, as it obviously depends on the application.

Experience Replay has been employed as a way to fight the problem of catastrophic forgetting, so it has been shown that it has some of the highest metrics in recalling information from previous tasks. Retraining everything from YOLO26n in a "brute force" approach has shown to have the highest mAP50-95 on new environments, so if the end goal is to have the most performant model, even at the cost of at least 5x the training time, this is the best way to go about it. Finally, in a surprising fashion, all three approaches output "flat" results when dealing with their ability to navigate and perform detection on the just seen environment. This can seem like a small consideration, but

registering that a model that took 76.2 minutes to train has a performance gap that can be discarded as noise from a model that took just 2.8, for a factor of over 27x, is a notable result.

Figure 6.4 is a classical Pareto Curve that allows for a clear and graphical understanding of the dominance of FT and CL when the training times are taken into consideration: in both applications the global maximum is produced by the JR method, but results of both the other method either are comparable to or they actually dominate the heavier process.

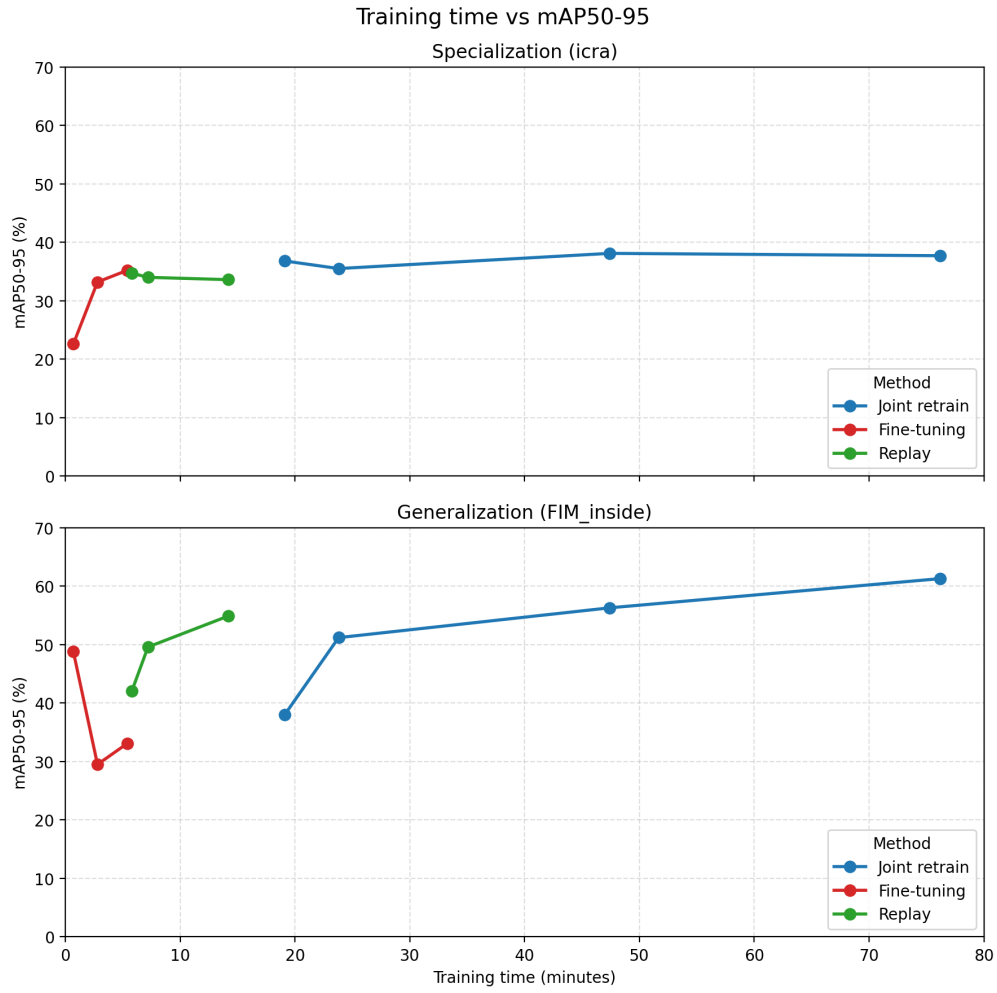


Figure 6.4: *Graphical demonstration of generalization and specialization with respect to the required time for training.*

6.5 Sequential Training

With all the explained experiments that has been performed, one notable limitation of this project is the absence of multiple iterations of learning, due to the never ending lack of data. This is relevant, as catastrophic forgetting tends to be clearer and to present its effect after the continuous employment of continual learning systems on multiple iterations. To perform an analysis on this particular situation, it has been decided to look into the result of a sequential and separated learning based on singular sessions of acquired data, simulating the continuous addition of images. The concept is graphically expressed in Figure 6.5:

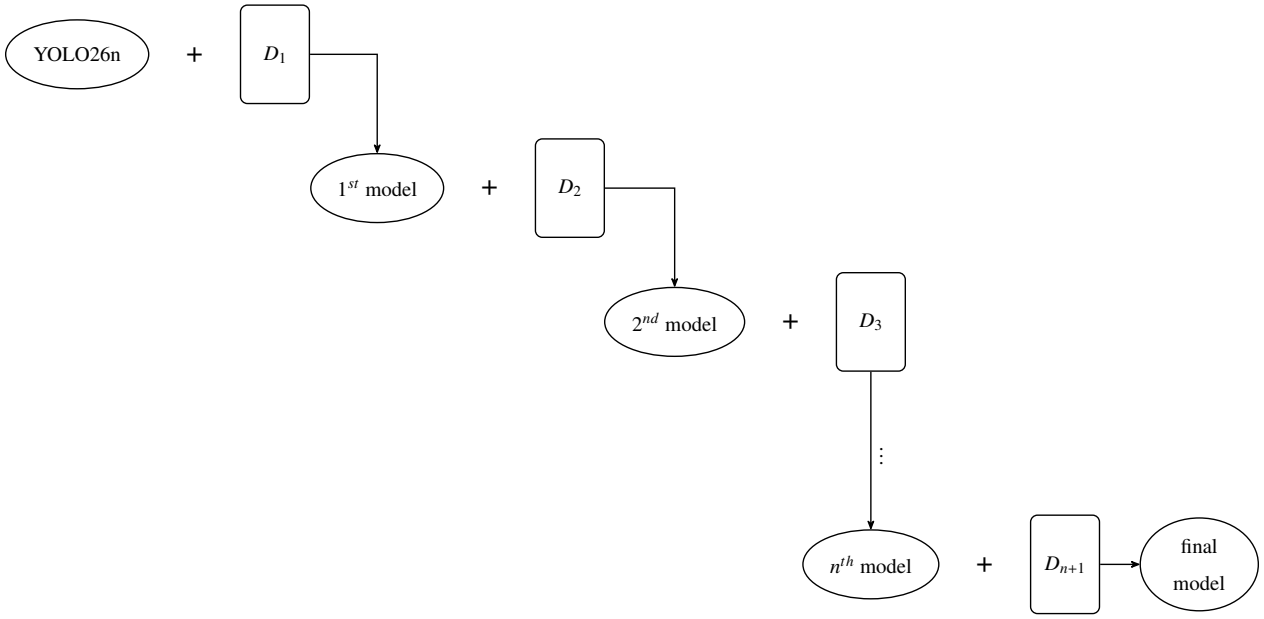


Figure 6.5: *Graphical explanation of the sequential training mechanism.*

To get useful answers from this addition to the research, every model at each step has been benchmarked not only on the three that seen before, that in this scenario lose all the semantic reason, but on holdouts from every session. A brief explanation for the logic behind this choice can be found in Figure 6.6.

The actual results can be found in the Table 6.3 and, before reading them, it is important to add a clarification about the terminology applied. It has been decided to use FT for Fine Tuning and CL for Continual Learning, whose most appropriate application in this work has been Fine Tuning with Experience Replay.

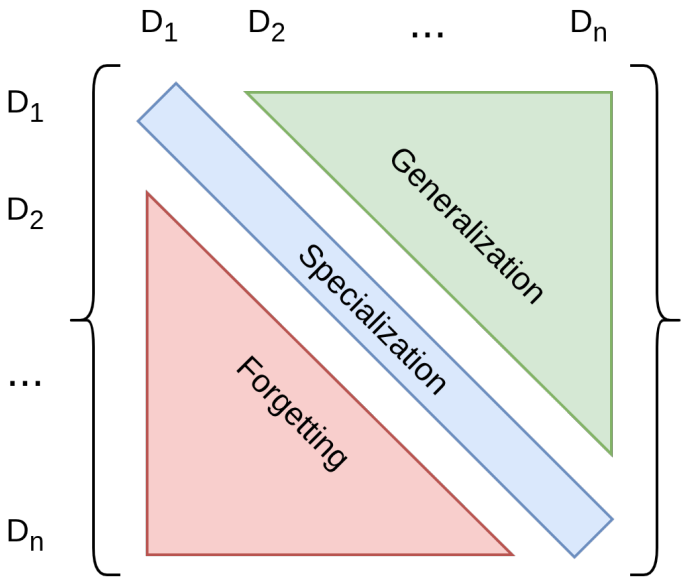


Figure 6.6: *Graphical explanation of the sequential results.*

	benchmarked on (CL / FT)				**never seen**
trained on	B_1: public	B_2: squeaky	B_3: FIM_out	B_4: icra	GEN:FIM_inside
D_1: public	**1.01 / 1.01**	.44 / .44	.57 / .57	.03 / .03	.99 / .99
D_2: squeaky	1.00 / .94	**1.03 / 1.02**	.85 / .82	.11 / .16	.98 / .92
D_3: FIM_out	.99 / .94	1.01 / .92	**.96 / .95**	.11 / .08	.98 / .93
D_4: icra	.99 / .75	.99 / .71	.90 / .68	**.82 / .97**	.88 / .56

Table 6.3: *Sequential Training Campaign mAP50-95 results, normalized per column with respect to the joint retrain as theoretical upper bound.*

The results here are what could have been expected, as it shows the creeping effects of catastrophic forgetting: reading the first column, the one referring to the `public` dataset (f1tenth + huawei) is the clearest one. It starts from very high mAP50-95 values, but at each successive step of sequential training, it takes a hit, degrading its capabilities more and more. This test is also able to show using facts the fact that the employment of Experience Replay has a big role in limiting said forgetting. Everything above the diagonal, and the final column, for readability's sake, shows instead that the generalization stays the same all throughout the successive step, and even takes a hit when trained on `icra`. This session causes such a level of drop in performance as it is a very different context than every other domain: it is much noisier and blurred. However, the fact that the generalization is able to stay pretty much stable, is an important consideration, as it is informative towards the fact that it is possible to add new knowledge to a model and it keeps its ability to generalize.

7. Future Works

7.1 Labelling

The main thing that still does not allow the proposed work to be self-sufficient is the requirement for human interaction while labeling new incoming data. It is possible, by design of the framework, to employ the latest or best weights to perform the labeling stage automatically, but it would be possible to stumble upon two main problems: confirmation bias [1] and model collapse [38]. The first refers to the risk for the model to become more and more sure of its own mistakes, as it keeps training on new data that possibly contains false positive or false negative predictions; the second stems from the same problematic concept, but concludes that the model not only loses the ability of learning new information, but this recursive approach may imply a great loss in generalization on new and real data. To avoid these problems, but still walking toward a self-sufficient final solution, two branches to improve the project are clear:

- **Auto-labeling on new environments**: if the final objective of the addition of new data is the on-the-fly specialization to the characteristics of a new environment, an approach could be to generate pseudo-labels and fine tune with experience replay, as seen before, to improve the abilities. This approach still suffers from confirmation bias and model collapse, but they can be limited by the employment of weakly or self supervised learning with confidence and consistency thresholds [39, 40]. This branch of development aims to allow the model to specialize on a new environment but in a much more controlled and less destructive way.
- **Self-supervised learning on edge devices**: this second development idea would be more suited for a long term approach to the entire framework. Self-supervised learning [14] does not require by design the presence of labels at all: it infers useful representation of the object it wants to detect in different ways, like geometric or analyzing pixel distributions in the image itself. By virtue of the lack of dependency from the labels and consequently, from a human or an error-prone approach, this approach would be particularly well suited for deployment of the framework on real-world systems, like edge devices. The main open challenge remains the computational cost of self-supervised training and inference relative to the limited resources

of embedded hardware.

7.2 On-edge Deployment

This work has analyzed the accuracy and the risk of false positive detection for the computed weights, that originated from the training that used as a starting point a YOLO26n model. This can be well-suited for the current deployment platform [12], a RoboRacer vehicle, as it runs on NVIDIA Jetson Orin: the lack of testing on the actual device due to time constraints, however, does not allow for conclusive considerations on the matter and represents itself a more than natural addition to the proposed work.

As shortly mentioned in the previous paragraph, this framework could have as an end goal the deployment on edge devices, with even lower computational capabilities [30]: to study if and how it is even possible would be an interesting field of work and a possible gap in the research landscape. This would require creating a custom Neural Network, created by keeping in mind the real constraint of the target device and the requirements for keeping the entire system running. Finally, it would be vital to be able to work toward an optimization of the functionalities of this work [36].

The end goal could be self-improving edge devices, able to specialize themselves and their abilities in the context of new environments, while keeping track of its ability to generalize on never-seen situations and still remembering past learned information.

8. Conclusions

Autonomous racing applications depend more and more on perception systems that rely on the employment of data that comes, more likely than not, from continuous streams of new data rather than being able to use a fixed dataset. This thesis focused on the development of a framework, together with the acquisition of a custom dataset, with the objective of the implementation of a camera-base object detection model, specifically analyzing the best approach for data addition.

The work was based on the conceptual fundament of continual learning, and adapted the design of everything else to follow the baselines of transparency, incrementability and ease of use: to use the tool it is not necessary to have a technical understanding of the implementation choices, nor it is mandatory to be able to write code. It is possible to conduct a study by just writing the experiments by filling a fixed structure, and, if needed, to add sessions by just adding them to a central folder and copying the path in a file.

This framework has been employed to give an answer to the previously stated final objective, and three strategies have been investigated: joint retraining, fine tuning and fine tuning with experience replay. All of these were benchmarked to illustrate their performance in three main fields, that are specialization, generalization and recalling on real racing scenarios.

The key contribution of this thesis is a novel and practical exploration of continual object detection, keeping in mind real-time and edge-orientation requirements, generating an intersection of research field mostly absent in the literature. The computed results allow to see that, between the strategies observed, and considering the computational cost along with the raw performances, experience replay, or continual learning, achieves the highest mean mAP50-95 on past information. To choose between the three possibilities in the fields of specialization and generalization is a matter of needs, as one can favor a global maximum at the cost of high training time, while another can show great efficiency in training but reach sub-optimal, but still comparable, results.

In conclusion, this thesis demonstrated the viability of the created framework as a starting platform to conduct further studies on the central topic of the study, continual learning. The performed research has also shown that, in this type of application, massive training campaigns on a huge amount of data are not much better than lighter ones, and it is possible with simple approaches

to limit the possible downsides of leveraging them, thus justifying the employment of continual learning techniques in such use cases.

Bibliography

[1] Eric Arazo et al. “Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning”. In: *International Joint Conference on Neural Networks (IJCNN)*. 2020, pp. 1–8. DOI: 10.1109/IJCNN48605.2020.9207304.

[2] Lubna Aziz et al. “Exploring deep learning-based architecture, strategies, applications and current trends in generic object detection: A comprehensive review”. In: *Ieee Access* 8 (2020), pp. 170461–170495.

[3] Israel Charles, Hossein Maghsoumi, and Yaser Fallah. “Advancing autonomous racing: A comprehensive survey of the RoboRacer (F1TENTH) platform”. In: *2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*. IEEE. 2025, pp. 207–213.

[4] Arslan Chaudhry et al. “Riemannian Walk for Incremental Learning: Understanding Forgetting and Intransigence”. In: *European Conference on Computer Vision (ECCV)*. 2018, pp. 532–547. DOI: 10.1007/978-3-030-01252-6_33.

[5] Matthias De Lange et al. “A continual learning survey: Defying forgetting in classification tasks”. In: *IEEE transactions on pattern analysis and machine intelligence* 44.7 (2021), pp. 3366–3385.

[6] Riccardo De Monte, Davide Dalle Pezze, and Gian Antonio Susto. “Teach YOLO to Remember: A Self-Distillation Approach for Continual Object Detection”. In: *Proceedings of the 4th Conference on Lifelong Learning Agents (CoLLAs)*. Vol. 330. 2025, pp. 487–499.

[7] Tausif Diwan, G Anirudh, and Jitendra V Tembhurne. “Object detection using YOLO: challenges, architectural successors, datasets and applications”. In: *multimedia Tools and Applications* 82.6 (2023), pp. 9243–9275.

[8] Arthur Douillard and Timothée Lesort. “Continuum: Simple Management of Complex Continual Learning Scenarios”. In: *arXiv preprint arXiv:2102.06253* (2021).

[9] Di Feng et al. “Deep multi-modal object detection and semantic segmentation for autonomous driving: Datasets, methods, and challenges”. In: *IEEE Transactions on Intelligent Transportation Systems* 22.3 (2020), pp. 1341–1360.

[10] Tao Feng, Mang Wang, and Hangjie Yuan. “Overcoming Catastrophic Forgetting in Incremental Object Detection via Elastic Response Distillation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 9427–9436.

[11] Robert M French. “Catastrophic forgetting in connectionist networks”. In: *Trends in cognitive sciences* 3.4 (1999), pp. 128–135.

[12] Francesco Gatti et al. “A Systematic Assessment of Embedded Neural Networks for Object Detection”. In: *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2020. DOI: 10.1109/ETFA46521.2020.9212039.

[13] Ian Goodfellow et al. *Deep learning*. Vol. 1. 2. MIT press Cambridge, 2016.

[14] Jean-Bastien Grill et al. “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 21271–21284.

[15] Glenn Jocher et al. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. 2026. DOI: 10.48550/arXiv.2606.03748. arXiv: 2606.03748 [cs.CV]. URL: https://arxiv.org/abs/2606.03748.

[16] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics. Accessed: 2026-09-15. 2023.

[17] James Kirkpatrick et al. “Overcoming catastrophic forgetting in neural networks”. In: *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.

[18] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 25. 2012, pp. 1097–1105.

[19] Zhizhong Li and Derek Hoiem. “Learning without forgetting”. In: *IEEE transactions on pattern analysis and machine intelligence* 40.12 (2017), pp. 2935–2947.

[20] Liyang Liu et al. “IncDet: In Defense of Elastic Weight Consolidation for Incremental Object Detection”. In: *IEEE Transactions on Neural Networks and Learning Systems* 32.6 (2021), pp. 2306–2319. DOI: 10.1109/TNNLS.2020.3002583.

[21] Yaoyao Liu et al. “Continual Detection Transformer for Incremental Object Detection”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.

[22] Yuyang Liu et al. “Augmented box replay: Overcoming foreground shift for incremental object detection”. In: *2023 IEEE/CVF international conference on computer vision (ICCV)*. IEEE. 2023, pp. 11333–11343.

[23] Vincenzo Lomonaco et al. “Avalanche: An End-to-End Library for Continual Learning”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2021, pp. 3600–3610.

[24] David Lopez-Paz and Marc’Aurelio Ranzato. “Gradient episodic memory for continual learning”. In: *Advances in neural information processing systems* 30 (2017).

[25] Arun Mallya and Svetlana Lazebnik. “Packnet: Adding multiple tasks to a single network by iterative pruning”. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE. 2018, pp. 7765–7773.

[26] Angelo G Menezes et al. “Continual object detection: a review of definitions, strategies, and challenges”. In: *Neural networks* 161 (2023), pp. 476–493.

[27] Fabrice Normandin et al. “Sequoia: A Software Framework to Unify Continual Learning Research”. In: *arXiv preprint arXiv:2201.13300* (2022).

[28] Rafael Padilla, Sergio L Netto, and Eduardo AB Da Silva. “A survey on performance metrics for object-detection algorithms”. In: *2020 international conference on systems, signals and image processing (IWSSIP)*. IEEE. 2020, pp. 237–242.

[29] Scott Drew Pendleton et al. “Perception, planning, control, and coordination for autonomous vehicles”. In: *Machines* 5.1 (2017), p. 6.

[30] Mateusz Piechocki, Alessandro Capotondi, and Marek Kraft. “Empowering On-Device Model Adaptation with an Edge AI Inference Accelerator”. In: *arXiv preprint arXiv:2607.18101* (2026).

[31] Sylvestre-Alvise Rebuffi et al. “iCaRL: Incremental Classifier and Representation Learning”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 2001–2010. DOI: 10.1109/CVPR.2017.587.

[32] Joseph Redmon and Ali Farhadi. “YOLO9000: Better, Faster, Stronger”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 6517–6525. DOI: 10.1109/CVPR.2017.690.

[33] Joseph Redmon and Ali Farhadi. “YOLOv3: An Incremental Improvement”. In: *arXiv preprint arXiv:1804.02767* (2018).

[34] Joseph Redmon et al. “You only look once: Unified, real-time object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788.

[35] Shaoqing Ren et al. “Faster R-CNN: Towards real-time object detection with region proposal networks”. In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (2016), pp. 1137–1149.

[36] Manuele Rusci et al. “Leveraging Automated Mixed-Low-Precision Quantization for Tiny Edge Microcontrollers”. In: *IoT Streams for Data-Driven Predictive Maintenance and Embedded Machine Learning (ITEM Workshop, ECML/PKDD)*. 2020, pp. 296–308. DOI: 10.1007/978-3-030-66770-2_22.

[37] Konstantin Shmelkov, Cordelia Schmid, and Karteek Alahari. “Incremental Learning of Object Detectors without Catastrophic Forgetting”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 3400–3409. DOI: 10.1109/ICCV.2017.368.

[38] Ilia Shumailov et al. “AI Models Collapse When Trained on Recursively Generated Data”. In: *Nature* 631.8022 (2024), pp. 755–759. DOI: 10.1038/s41586-024-07566-y.

[39] Kihyuk Sohn et al. “FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 596–608.

[40] Antti Tarvainen and Harri Valpola. “Mean Teachers are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 1195–1204.

[41] Gido M Van de Ven and Andreas S Tolias. “Three scenarios for continual learning”. In: *arXiv preprint arXiv:1904.07734* (2019).

[42] Liyuan Wang et al. “A comprehensive survey of continual learning: Theory, method and application”. In: *IEEE transactions on pattern analysis and machine intelligence* 46.8 (2024), pp. 5362–5383.

[43] Shizhou Zhang et al. “YOLO-IOD: Towards Real-Time Incremental Object Detection”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 15. 2026, pp. 12744–12752. DOI: 10.1609/aaai.v40i15.38271.